

2025/26

U.D.2. CREACIÓN DE INTERFACES WEB HOMOGÉNEAS.

2.4 Propiedades CSS – Parte 1.





ÍNDICE

1. Unidades de medida	4
1.1 Unidades absolutas	4
1.2 Unidades relativas	4
1.3 Usos de unidades	5
2. Tamaño de los elementos	6
2.1 ¿Qué son las propiedades CSS width y height?	6
2.2 Cómo usar las propiedades CSS width y height	7
2.3 ¿Qué son las propiedades CSS max-width y max-height?	7
2.4 ¿Qué son las propiedades CSS min-width y min-height?	8
2.5 Cómo usar las propiedades CSS max-width, min-width, max-height y min-height	8
3. Media queries para cumplir con las necesidades del diseño responsive	10
3.1 Media query indicando el ancho mínimo de pantalla	10
3.2 Media query indicando el ancho máximo de pantalla	11
3.3 Media query combinando múltiples condiciones	11
3.4 Más ejemplos para definir el ancho del dispositivo	11
3.5 Otras propiedades o condiciones	13
3.6 Operadores lógicos en media queries	14
3.6.1 Operador AND	14
3.6.2 Operador NOT	14
3.6.3 Operador OR	15
3.6.4 Operador ONLY	15
3.7 Mejoras de sintaxis en Media Queries	15
4. El color	17
4.1 Colores con palabras reservadas o clave	17
4.2 Colores con RGB (decimal, hexadecimal y porcentual)	17
4.3 Colores con HSL	18
4.4 Colores con transparencia	19
5. El diseño del fondo	21
5.1 Color de fondo ('background-color')	21
5.2 Imagen de fondo ('background-image')	22
5.2.1 Fondo con degradados	23
5.2.1.1 Gradiente lineal ('linear-gradient')	23



5.2.1.2	Degradado radial ('radial-gradient')	25
5.2.1.3	Degradado cónico ('conic-gradient')	28
5.2.1.4	Generadores de gradientes	29
5.3	Repetición y posición	30
5.3.1	background-repeat	30
5.3.2	background-position	31
5.4	background-size	33
5.5	Opacity	35
6.	Efecto Parallax	36
6.1	Pasos para crear un efecto parallax	36
6.1.1	Paso 1: crear los contenedores en HTML	36
6.1.2	Paso 2: agregar las propiedades CSS	37
6.2	Ejemplo sencillo	38
7.	Propiedades de texto	41
7.1	Interlineado de texto ('line-height')	41
7.2	Alineado de texto ('text-align')	42
7.3	Decoración de texto ('text-decoration')	42
7.4	Alineación vertical ('vertical-align')	45
7.5	Transformación del texto ('text-transform')	47
7.6	Letter spacing	48
7.7	Word spacing	49
7.8	Tabulado del texto ('text-indent')	50
7.9	Sombra del texto ('text-shadow')	50
7.10	Sombra a elementos caja ('box-shadow')	52
7.10.1	Generadores de sombra online	53
8.	Propiedades de Fuente	54
8.1	Fuente de letra ('font-family')	54
8.2	Tamaño de letra ('font-size')	55
8.3	Estilo de letra ('font-style')	57
8.4	Grosor de letra ('font-weight')	58
8.5	Variante de fuente ('font-variant')	60
8.6	La propiedad fundamental 'font'	60
8.7	Fuentes externas	62
8.7.1	Fuentes web online	65
8.7.2	Convertidores de fuentes online	65



9. Hyphens CSS	66
9.1 Sintaxis y valores	66
9.2 Definir el idioma para una correcta separación de sílabas	67
9.3 Fragmentos de texto en otro idioma al predefinido	67
9.4 Ejemplos de uso	68
9.4.1 Usando hyphens en cards	68
9.4.2 Usando hyphens con line-clamp	69
10. Display	71
11. Posicionar o Position	73
11.1 Static	73
11.2 Relative	74
11.3 Fixed	75
11.4 Absolute	76
11.5 Sticky	78
12. Float y Clear	79
12.1 Problemas con flotantes en CSS y sus soluciones	81
13. Propiedades de visibilidad	84
13.1 'z-index'	84
13.2 'overflow'	85
13.3 'visibility'	86



1. Unidades de medida

Como ya hemos visto, el tamaño de los elementos de una web se puede expresar en unidades **absolutas** y **relativas**.

1.1 Unidades absolutas

Las **unidades absolutas** mantienen su aspecto y se visualizan siempre igual independientemente de las características del dispositivo.

Unidad	Descripción
in	Pulgadas (1 pulgada = 2.54 cm)
cm	Centímetros
mm	Milímetros
pt	Puntos (1 pt = 1/72 pulgadas)
pc	Picas (1 pica = 12 puntos)

1.2 Unidades relativas

Las **unidades relativas** se ajustan a cada tipo de dispositivo ya que dependen de la resolución de cada pantalla.

Unidad	Descripción
px	Píxeles (relativo al dispositivo). Es la unidad más utilizada y representa un punto en la pantalla. Se usa para tamaños fijos y proporciona control preciso sobre el diseño.
em	Relativo al tamaño de la fuente del elemento (2 em significa 2 veces el tamaño de la fuente actual).
%	Porcentaje (relativo al elemento padre). Es útil para hacer diseños fluidos y responsivos teniendo en cuenta la relación de los elementos con su contenedor padre .
vh y vw	Representan un porcentaje del ancho y alto de la ventana del navegador , respectivamente (medidas relativas de acuerdo al viewport). 1vh = 1% de la altura del viewport. 100vh = altura del viewport. Son útiles para crear diseños responsive basados en el tamaño de la pantalla .



fr	Flexible Grid Units (fr). Se utiliza en Grid Layout y representa una fracción del espacio disponible en un contenedor . Es útil para distribuir el espacio disponible entre elementos flexibles.
-----------	---

1.3 Usos de unidades

Normalmente es **recomendable usar unidades relativas** en la medida de lo posible, ya que mejora la accesibilidad de la página web y permite que los documentos se adapten fácilmente a cualquier medio. Por tanto, **para la creación de una página web, el uso de medidas absolutas queda descartado**.

Aunque no hay un criterio definido, el organismo **W3C**, **recomienda** el uso de la unidad **em** para indicar el tamaño del texto. **El tamaño de los ems se establece en base al tamaño que tenga definido el navegador**.

Existe también la unidad **REM (rem)** que es similar al em, pero se basa en el tamaño de fuente del elemento raíz (generalmente el tamaño de fuente del elemento HTML).

Usualmente **el tamaño de una fuente por defecto en los navegadores es de 16px**. Por tanto, tendríamos que **16px = 1em** y podríamos definir la siguiente conversión entre unidades.

px	em	%
12	0,750	75
14	0,875	87,5
16	1,000	100
18	1,125	112,5
20	1,250	125

Hay que tener en cuenta que el tamaño base definido en los navegadores puede ser modificado por los usuarios.



2. Tamaño de los elementos

En este apartado vamos a aprender cómo usar las **propiedades CSS** *width*, *height*, *min-width*, *max-height* y *min-height* para controlar el tamaño de los elementos en una página web.

Veamos una tabla resumen de las propiedades que vamos a tratar en este apartado:

Nombre propiedad	Descripción	Valores
<i>width</i>	Establece el ancho del área de contenido de un elemento	Unidad de longitud (px, em, %, etc.), auto, initial, inherit
<i>height</i>	Establece el alto del área de contenido de un elemento	Unidad de longitud (px, em, %, etc.), auto, initial, inherit
<i>max-width</i>	Establece el ancho máximo que puede tener un elemento	Unidad de longitud (px, em, %, etc.), none, initial, inherit
<i>min-width</i>	Establece el ancho mínimo que debe tener un elemento	Unidad de longitud (px, em, %, etc.), 0, initial, inherit
<i>max-height</i>	Establecer el alto máximo que debe tener un elemento	Unidad de longitud (px, em, %, etc.), 0, initial, inherit
<i>min-height</i>	Establecer el alto mínimo que debe tener un elemento	Unidad de longitud (px, em, %, etc.), 0, initial, inherit

2.1 ¿Qué son las propiedades CSS *width* y *height*?

Las **propiedades CSS** *width* y *height* se usan para **establecer el ancho y el alto de un elemento**. Por defecto, estas propiedades **se refieren al área de contenido** del elemento, es decir, el espacio que ocupa el texto, las imágenes u otros elementos dentro del elemento. El área de contenido **no incluye el padding (relleno), el border (borde) ni el margin (margen) del elemento**.

Sin embargo, podemos cambiar este comportamiento con la propiedad CSS *box-sizing*, que nos permite definir qué partes del elemento se incluyen en el cálculo del ancho y el alto.



2.2 Cómo usar las propiedades CSS width y height

Veamos algunos ejemplos de uso de *width* y *height* con diferentes unidades:

- Para darle un **ancho y un alto fijo a un elemento**, podemos usar **píxeles** como unidad. Por ejemplo, si queremos que un elemento tenga un ancho de 200px y un alto de 100px, podemos usar este código:

```
div {  
    width: 200px;  
    height: 100px;  
}
```

- Para darle un **ancho y un alto relativo al elemento contenedor**, podemos usar **porcentajes** como unidad. Por ejemplo, si queremos que un elemento ocupe el 50% del ancho y el 25% del alto del elemento padre, podemos usar este código:

```
div {  
    width: 50%;  
    height: 25%;  
}
```

2.3 ¿Qué son las propiedades CSS max-width y max-height?

Las propiedades CSS *max-width* y *max-height* se usan para establecer el ancho y el alto máximo de un elemento. Estas propiedades nos permiten limitar el tamaño de un elemento para que no supere cierto valor, incluso si le damos un ancho o un alto mayor con las propiedades *width* o *height*.

Esto puede ser muy útil para crear **diseños responsivos** que se adapten a diferentes tamaños de pantalla. Por ejemplo, podemos usar *max-width* para evitar que una imagen se salga del contenedor o que se distorsione al cambiar la resolución. También podemos usar *max-height* para controlar la altura de un elemento que tenga contenido variable, como un menú desplegable o una caja de texto.



2.4 ¿Qué son las propiedades CSS min-width y min-height?

Las propiedades CSS *min-width* y *min-height* se utilizan para establecer la anchura y la altura mínima de un elemento, respectivamente. Estas propiedades impiden que el elemento se reduzca a un ancho o alto menor que el valor especificado.

Las propiedades *min-width* y *min-height* se pueden utilizar para **garantizar que un elemento tenga un tamaño mínimo**, independientemente del tamaño del contenido que contenga. Por ejemplo, se pueden utilizar para garantizar que un elemento siempre tenga un tamaño mínimo de visualización, incluso en pantallas pequeñas.

2.5 Cómo usar las propiedades CSS max-width, min-width, max-height y min-height

Veamos algunos ejemplos de uso de *max-width*, *min-width*, *max-height* y *min-height* con diferentes unidades:

- 1) Para darle un **ancho y un alto máximo** a un elemento en **píxeles**, podemos usar esta unidad como valor. Por ejemplo, si queremos que una imagen no supere los 300px de ancho ni los 200px de alto, podemos usar este código:

```
img {  
    max-width: 300px;  
    max-height: 200px;  
}
```

- 2) Para darle un **ancho y un alto máximo** a un elemento en **porcentajes**, podemos usar esta unidad como valor. Por ejemplo, si queremos que una imagen no ocupe más del 50% del ancho ni del 25% del alto del elemento contenedor, podemos usar este código:

```
img {  
    max-width: 50%;  
    max-height: 25%;  
}
```



3) El siguiente código establece el **ancho máximo** del elemento **div** a 300 **píxeles**:

```
div {  
    max-width: 300px;  
}
```

4) El siguiente código establece el alto mínimo del elemento **div** a 100 píxeles:

```
div {  
    min-height: 100px;  
}
```

5) **Veamos un ejemplo práctico.** Para crear un contenido a pantalla completa que no se haga más grande al llegar a un ancho máximo, podemos usar la propiedad **max-width** junto con la propiedad **width**. Así, le damos al elemento un ancho relativo al tamaño de la pantalla, pero también le ponemos un límite absoluto.

Por ejemplo, si queremos que el contenido ocupe el 95% del ancho de la pantalla, pero que no supere los 1200px, podemos usar este código:

```
.content {  
    width: 95%; /* El contenido ocupa el 95% del ancho de la pantalla */  
    max-width: 1200px; /* El contenido no ocupará más de 1200px */  
    margin: 0 auto; /* Centrar el contenido */  
}
```

De esta forma, el contenido se adaptará al tamaño de la pantalla, pero no se hará más grande de lo necesario. Esto puede mejorar la legibilidad y el diseño de tu página web.

3. Media queries para cumplir con las necesidades del diseño responsive

Los diseños responsive o diseño web adaptable o adaptativo se lleva a cabo mediante la utilización de **@media** junto con las propiedades **min-width** (anchura mínima) o **max-width** (anchura máxima). El objetivo del diseño responsive es **adaptar la apariencia de las páginas web al dispositivo** que se esté utilizando para visitarlas.

3.1 Media query indicando el ancho mínimo de pantalla

A continuación, se muestra un **ejemplo de diseño responsive** con **@media** y la propiedad **min-width**, tomando como referencia el tamaño promedio de las pantallas que existen actualmente.

```
body{
    background: orange;
}
div{
    width: 80%;
    height: 150px;
    margin: 8%;
    display: inline-block;
    background: white;
}

@media screen and (min-width: 480px){
    body{
        background: #34A647;
    }
}
```

Se han establecido las dimensiones partiendo de un diseño **MobileFirst**, que consiste en diseñar primero para dispositivos móviles debido al aumento considerable de este tipo de dispositivos. **Es más fácil rellenar que limpiar.**

A partir de 480px (aprox.), tenemos un móvil en posición horizontal. 480px es un **punto de ruptura**.

```
@media screen and (min-width: 767px){
    body{
        background: #34A2A6;
    }
    div{
        width: 40%;
        margin: 4%;
    }
}
```

A partir de 767px (aprox.), tenemos una tablet. 767px es otro **punto de ruptura**.

Para el diseño responsive, se recomienda siempre utilizar %

```
@media screen and (min-width: 950px){
    body{
        background: #A76A9E;
    }
    div{
        width: 28%;
        margin: 2%;
    }
}
```

A partir de 950px (aprox.), tenemos una pantalla de escritorio. 950px es otro **punto de ruptura**.



3.2 Media query indicando el ancho máximo de pantalla

Todos los estilos que se incluyan en el interior de la siguiente media query serán utilizados en pantallas que tengan un ancho máximo de 767px (punto de ruptura de 767px).

```
@media (max-width: 767px) {  
    /* Estilos para pantallas con un ancho máximo de 768px */  
}
```

3.3 Media query combinando múltiples condiciones

Es posible combinar múltiples condiciones en una media query para aplicar estilos solo cuando se cumplan todas las condiciones. Todos los estilos que se incluyan en el interior de la siguiente media query serán utilizados en pantallas que tengan un ancho entre 768px y 1024px (puntos de ruptura de 768px y 1024px).

```
@media (min-width: 768px) and (max-width: 1024px) {  
    /* Estilos para pantallas con un ancho entre 768px y 1024px */  
}
```

3.4 Más ejemplos para definir el ancho del dispositivo

- 1) Supongamos que queremos aplicar un estilo diferente en pantallas de ancho superior a 1024px, inferior a 1024px e inferior a 480px, tendríamos las siguientes *media queries*.

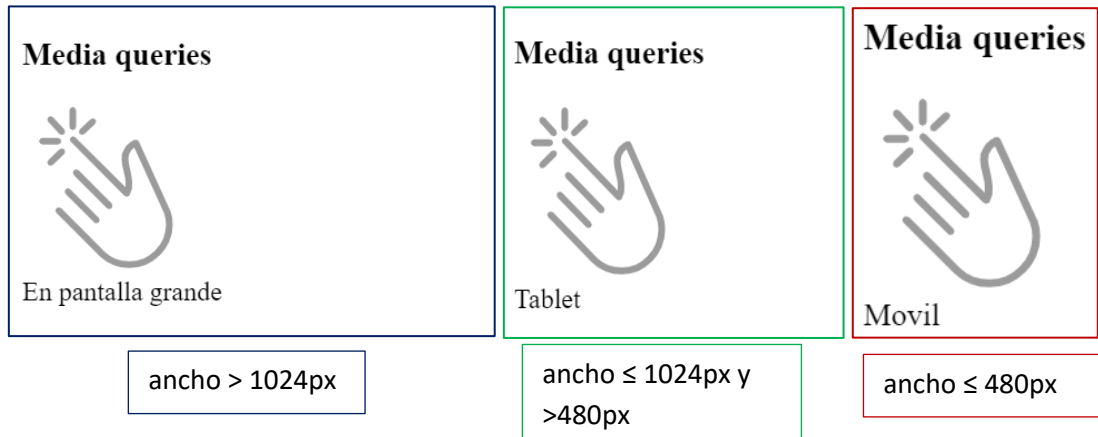
Código **CSS**:

```
.click:after {  
    content:"En pantalla grande";  
}  
@media (max-width: 1024px) {  
    .click:after {  
        content:"Tablet";  
    }  
}  
@media (max-width: 480px) {  
    .click:after {  
        content:"Movil";  
    }  
}
```



Código HTML:

```
<h3>Media queries</h3>
<div class="click">
  <br>
</div>
```



- 2) La primera *media query* especifica una hoja de estilo para ser utilizada cuando la ventana tiene un ancho entre 450 y 800 píxeles y la segunda *media query* para pantallas menores a 449 píxeles.

Código CSS:

```
.click:after {
  content:"En pantalla grande";
}
@media (min-width: 450px) and (max-width: 800px) {
  .click:after {
    content:"Tablet";
  }
}

@media (max-width: 449px) {
  .click:after {
    content:"Movil";
  }
}
```



Código **HTML**:

```
<h3>Media queries</h3>
<div class="click">
  <br>
</div>
```

Media queries	Media queries	Media queries
		
En pantalla grande	Tablet	Movil
ancho > 800px	ancho ≤ 800px y ≥450px	ancho < 450px

3.5 Otras propiedades o condiciones

Además del ancho hay muchas otras propiedades que se pueden utilizar para aplicar o no estilos. Las siguientes son algunas de ellas:

Nombre	Descripción
<i>width</i>	Anchura del <i>viewport</i>
<i>height</i>	Altura del <i>viewport</i>
<i>aspect-ratio</i>	Relación de aspecto anchura-altura del <i>viewport</i>
<i>orientation</i>	Orientación del <i>viewport</i>
<i>resolution</i>	Densidad de píxeles del dispositivo
<i>scan</i>	Proceso de escaneo del dispositivo
<i>grid</i>	Si el dispositivo es grid o bitmap
<i>update-frequency</i>	Velocidad de actualización del dispositivo para modificar la apariencia del contenido
<i>overflow-block</i>	Cómo maneja el dispositivo el contenido que excede los límites del <i>viewport</i> a lo largo del eje de bloque



<i>overflow-inline</i>	Cómo maneja el dispositivo el contenido que excede los límites del eje <i>inline</i>
<i>color</i>	Componente de número de bits por color del dispositivo, o cero si el dispositivo no es a color
<i>color-index</i>	Número de entradas en la tabla de búsqueda de color del dispositivo, o cero si el dispositivo no usa una tabla
<i>monochrome</i>	Bits por píxel en el buffer de marco monocromático del dispositivo, o 0 si el dispositivo no es monocromático
<i>hover</i>	Si se puede posicionar el puntero sobre los elementos

3.6 Operadores lógicos en media queries

Como hemos visto, las *media queries* nos permiten utilizar operadores lógicos para comprobar si se cumple una condición. Los principales son: **and**, **not**, **or** y **only**.

3.6.1 Operador AND

Cuando usamos el operador **and**, combinaremos un tipo de media (o soporte) con una o varias características de la media. Por ejemplo, el siguiente código aplicará estilos cuando la pantalla tenga un ancho mínimo de 700px y la orientación de la misma sea horizontal (*landscape*).

```
@media (min-width: 700px) and (orientation: landscape) { ... }
```

3.6.2 Operador NOT

El operador **not** invierte el significado de toda una consulta de medios. Por ejemplo, el siguiente código aplicará estilos cuando no se cumpla la condición especificada, es decir a cualquier elemento que no sea una pantalla convencional ni sea monocromo.

```
@media not screen and (monochrome) { ... }
```



3.6.3 Operador OR

Usaremos el operador **or** para probar una coincidencia entre más de una característica, resolviendo como true si alguna de las características es verdadera.

Por ejemplo, el siguiente código aplicará estilos cuando tengamos una pantalla monocromática o capacidad de desplazamiento.

```
@media (not (color)) or (hover) { ... }
```

3.6.4 Operador ONLY

El operador **only** evita que los navegadores web antiguos (que no admiten media queries con características de medios) apliquen los estilos especificados para todos los medios. **No tiene ningún efecto en los navegadores modernos.** Por ejemplo, el siguiente código aplicará estilos para un medio de tipo screen y si es a color.

```
@media only screen and (color) { ... }
```

3.7 Mejoras de sintaxis en Media Queries

La especificación Media Queries Level 4 incluye una nueva sintaxis de rangos para media queries que consiste en usar **operadores de comparación** (populares en cualquier lenguaje de programación) en lugar de min-width y max-width.

- < evalúa si un valor es **menor** que otro valor.
- > evalúa si un valor es **mayor** que otro valor.
- = evalúa si un valor es **igual** a otro valor.
- <= evalúa si un valor es **menor o igual** a otro valor.
- >= evalúa si un valor es **mayor o igual** a otro valor.

A continuación, se muestran algunos ejemplos con las equivalencias para las funcionalidades **max- y min-**

```
@media (max-width: 30em) { ... }
```



```
@media (width <= 30em) { ... }
```




```
@media (min-width: 30em) { ... }
```



```
@media (width => 30em) { ... }
```

```
@media (min-width: 30em) and  
(max-width: 50em) { ... }
```



```
@media (30em <= width <= 50em)  
{ ... }
```

La especificación Media Queries Level 4 tiene un soporte razonable en los navegadores modernos, pero **algunas características multimedia no son compatibles**.





4. El color

Uno de los primeros estilos en los que debemos pensar cuando diseñamos una interfaz web es en la paleta de colores que vamos a aplicar.

Los valores de los **colores** se pueden definir mediante su **nombre o palabra clave**, en **código hexadecimal** (#RRGGBBAA) o mediante sus valores en los siguientes **formatos**:

- **RGB** (Red, Green, Blue).
- **HSL** (Hue, Saturation, Lightness).
- **RGBA** (Red, Green, Blue, Alpha).
- **HSLA** (Hue, Saturation, Lightness, Alpha).

Todos los navegadores actuales soportan **140 nombres** de colores. La lista completa la puedes ver en la siguiente url: <https://htmlcolorcodes.com/es/nombres-de-los-colores/>

4.1 Colores con palabras reservadas o clave

Existen colores en CSS que de por sí ya tienen un **nombre propio**, es decir, existe una palabra reservada para cada uno de estos colores. Su sintaxis, utilizando la propiedad **color**, es la siguiente:

```
color: red;  
color: orange;  
color: tan;  
color: rebeccapurple;  
color: transparent;
```

4.2 Colores con RGB (decimal, hexadecimal y porcentual)

Para dar color a nuestra interfaz web, podemos utilizar el **sistema RGB** que ya hemos visto. El sistema **RGB** forma el color mediante la **mezcla de luces** roja, verde y azul. Por tanto, si no tenemos luz obtendremos el color negro, y si tenemos las tres luces antes mencionadas, obtendremos el color blanco.

A la hora de representar cada uno de los colores, es posible utilizar tanto el sistema de **numeración decimal** (0 a 255) como el **hexadecimal**.

En CSS, un color puede ser especificado en numeración **decimal** o en **porcentajes**, utilizando la siguiente sintaxis: **rgb(red, green, blue);** Además, también se puede expresar en **hexadecimal**.

A continuación, se muestra un ejemplo con la salida correspondiente.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .luzRGB-decimal {
      background-color: rgb(0, 255, 0);
    }
    .luzRGB-porcentajes {
      background-color: rgb(70%, 40%, 75%);
    }
    .luzRGB-hex {
      background-color: #55BB88;
    }
    .luzRGB-hex-reducido {
      background-color: #5B8;
    }
  </style>
</head>
<body>

<h1>Specify colors using RGB values</h1>

<h2 class="luzRGB-decimal">rgb(0, 255, 0)</h2>
<h2 class="luzRGB-porcentajes">rgb(70%, 40%, 75%)</h2>
<h2 class="luzRGB-hex">#55BB88</h2>
<h2 class="luzRGB-hex-reducido">#5B8</h2>

  </body>
</html>
```

Diagram illustrating the mapping of CSS color values to their visual representation:

- RGB en decimal: `rgb(0, 255, 0);`
- RGB en porcentajes: `rgb(70%, 40%, 75%);`
- RGB en hexadecimal: `#55BB88;` and `#5B8;`

Specify colors using RGB values

rgb(0, 255, 0)

rgb(70%, 40%, 75%)

rgb(25, 40%, 128)

#55BB88

#5B8

Cuando se repite un mismo dígito en una de las luces, se puede simplificar y poner un solo dígito.

4.3 Colores con HSL

Además de dar color a nuestra interfaz web mediante el sistema RGB, podemos darle color mediante el sistema **HSL**.

HSL es el acrónimo de **hue** (*matiz o tono*), **saturation** (*saturación*), y **lightness** (*brillo o luminosidad*). Por tanto, el color se formará en base a los valores que le demos a la tonalidad, a la saturación, y a la luz.



Su sintaxis es la siguiente: **hsl(hue, saturation, lightness);**

- **Hue:** es el grado en el nos situamos dentro del círculo cromático. 0º es rojo, 120º es verde, y 240º es azul.
- **Saturation:** es el porcentaje de saturación. 0% significa una sombra de gris, y 100% es el color completo o full color.
- **Lightness:** es el porcentaje de luz. 0% es negro y 100% es blanco.

A continuación, se presenta un ejemplo, con su correspondiente salida.

```
<!DOCTYPE html>
<html>
<style>
div {
  background-color: hsl(180, 50%, 50%);
  color: hsl(0, 0%, 100%);
  padding: 20px;
}
</style>
<body>

<div>

<p>London is the capital city of England.
It is the most populous city in the United Kingdom,
with a metropolitan area of over 13 million inhabitants.
</p>

</div>
</body>
</html>
```

London is the capital city of England. It is the most populous city in the United Kingdom, with a metropolitan area of over 13 million inhabitants.

4.4 Colores con transparencia

En CSS, podemos dar color usando los sistemas RGB y HSL vistos anteriormente, y además les podemos dar **opacidad o transparencia**.

⇒ **RGB con un canal alfa u opacidad:** **rgba (red, green, blue, alpha);**

Las luces rojo, verde y azul se definen como ya hemos visto, en porcentaje o en decimal de 0 a 255. Alpha define la opacidad y es un número entre **0.0** (totalmente transparente) y **1.0** (completamente opaco).



```
<!DOCTYPE html>
<html>
<head>
<style>
#p1 {background-color:rgba(255,0,0,0.3);}
#p2 {background-color:rgba(0,255,0,0.3);}
#p3 {background-color:rgba(0,0,255,0.3);}
#p4 {background-color:rgba(192,192,192,0.3);}
#p5 {background-color:rgba(255,255,0,0.3);}
#p6 {background-color:rgba(255,0,255,0.3);}
</style>
</head>
<body>

<h1>The rgba() Function</h1>

<p>RGB colors with opacity:</p>
<p id="p1">Red</p>
<p id="p2">Green</p>
<p id="p3">Blue</p>
<p id="p4">Grey</p>
<p id="p5">Yellow</p>
<p id="p6">Cerise</p>

</body>
</html>
```

The rgba() Function

RGB colors with opacity:



⇒ **HSL con un canal alfa u opacidad:**

hsla(hue, saturation, Lightness, alpha);

El tono, la saturación y el brillo se definen como ha hemos visto en el apartado anterior. Alpha define la opacidad y es un número entre **0.0** (totalmente transparente) y **1.0** (completamente opaco).

```
<!DOCTYPE html>
<html>
<head>
<style>
#p1 {background-color:hsla(120,100%,50%,0.3);}
#p2 {background-color:hsla(120,100%,75%,0.3);}
#p3 {background-color:hsla(120,100%,25%,0.3);}
#p4 {background-color:hsla(120,60%,70%,0.3);}
#p5 {background-color:hsla(290,100%,50%,0.3);}
#p6 {background-color:hsla(290,60%,70%,0.3);}
</style>
</head>
<body>

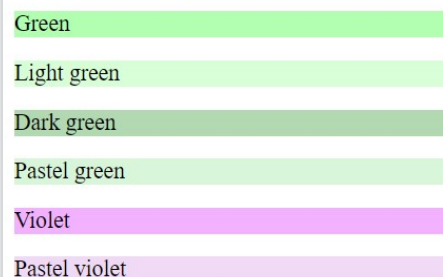
<h1>The hsla() Function</h1>

<p>HSL colors with opacity:</p>
<p id="p1">Green</p>
<p id="p2">Light green</p>
<p id="p3">Dark green</p>
<p id="p4">Pastel green</p>
<p id="p5">Violet</p>
<p id="p6">Pastel violet</p>

</body>
</html>
```

The hsla() Function

HSL colors with opacity:





5. El diseño del fondo

En cuanto al diseño del fondo, existen ciertas propiedades fundamentales para establecer el fondo de cada elemento.

Algunas de las propiedades relacionadas con el fondo más utilizadas son las siguientes:

Propiedad	Descripción	Valores
<i>color</i>	Color del texto	RGB HSL HEX nombre del color RGBA HSLA
<i>background-color</i>	Color de fondo	RGB HSL HEX nombre del color RGBA HSLA
<i>background-image</i>	Imagen de fondo	url("...") none
<i>background-repeat</i>	Repetición de la imagen de fondo	repeat repeat-x repeat-y no-repeat
<i>background-attachment</i>	Desplazamiento de la imagen de fondo	scroll fixed
<i>background-position</i>	Posición de la imagen de fondo	percentage length left center right
<i>background-size</i>	Tamaño de la imagen de fondo	valor
<i>opacity</i>	Transparencia de un elemento	[0 – 1] (0 → totalmente transparente)

5.1 Color de fondo ('background-color')

El color de fondo de un elemento en HTML se define mediante esta propiedad. Como ya se ha visto, el color puede expresarse mediante código hexadecimal, código RGB, sistema HSL o con palabra de color clave ya definidas. La palabra reservada, **transparent**, se utiliza para que el elemento tome el valor del elemento situado debajo de él, es decir, del que hereda.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    background-color: color | transparent | inherit ;  
}
```



Ejemplo:

```
p {  
  background-color: teal;  
  color: white;  
}  
  
h1 {  
  background-color: rgb(153,102,153);  
  color: rgb(255,255,204);  
}  
  
h3 {  
  background-color: #777799;  
  color: #FFFFFF;  
}
```

5.2 Imagen de fondo ('background-image')

La propiedad '**background-image**' permite utilizar una imagen como fondo de cualquier elemento. Si se utiliza esta propiedad, **la imagen quedará por encima del color establecido con la propiedad anterior**. Se recomienda indicar un color de fondo, para que sea este el que aparezca en el navegador si ocurre algún problema con la imagen seleccionada, de esta forma se evitarán apariencias indeseadas.

La **sintaxis** es la siguiente:

```
etiqueta html {  
  background-image: url | none | inherit ;  
}
```

Es recomendable crear una carpeta de imágenes que se encuentre en el mismo directorio que los archivos CSS y que almacene todas las imágenes utilizadas en el diseño de las páginas.

Si la imagen que se quiere mostrar es demasiado grande para el fondo del elemento, solamente se mostrará la parte de imagen comprendida en el tamaño del elemento. Si la imagen es más pequeña que el elemento, CSS la repetirá horizontalmente y verticalmente hasta llenar el fondo del elemento.



5.2.1 Fondo con degradados

Los **degradados CSS** permiten mostrar transiciones suaves entre dos o más colores especificados. Hay que tener especial cuidado con esta propiedad, ya que necesitará, en algunos casos, el **prefijo del navegador** para que sea compatible.

CSS especifica los siguientes **tres tipos** de degradados:

- 1) **Gradientes lineales** (van hacia abajo/arriba/izquierda/derecha/diagonalmente).
- 2) **Degradados radiales** (definidos por su centro).
- 3) **Degradados cónicos** (giran alrededor de un punto central)

5.2.1.1 Gradiente lineal ('linear-gradient')

Para crear un degradado lineal se deben definir al menos **dos topes de color** (o "color stop"). Los topes de color son los colores entre los que se quieren hacer transiciones suaves.

También se puede **establecer un punto de partida y una dirección** (o un ángulo) junto con el efecto de degradado.

Su **sintaxis** es la siguiente:

```
etiqueta html {  
    background-image: linear-gradient(direction|angle, color-  
        stop1, color-stop2, ...);  
}
```

```
etiqueta html {  
    background-image: linear-gradient(direction|angle, color-stop1  
        size, color-stop2 size, ...);  
}
```

```
etiqueta html {  
    background-image: linear-gradient(color-stop1, color-stop2, ...);  
}
```

Algunos de los valores de *direction* pueden ser:

- *to bottom*: de arriba a abajo. Es el valor por defecto.



- *to right*: de izquierda a derecha.
- *to bottom right*: en diagonal de arriba a la izquierda a abajo a la derecha.
- etc.

Si queremos un **mayor control** sobre la dirección del gradiente, podemos **definir un ángulo**, en lugar de las direcciones predefinidas (hacia abajo, hacia arriba, hacia la derecha, hacia la izquierda, hacia abajo a la derecha, etc.).

```
etiqueta html {  
    background-image: linear-gradient(angle, color-stop1, color-stop2, ...);  
}
```

Algunos de los valores de *angle* pueden ser:

- 0deg, que equivale a “hacia arriba”.
- 90deg, que equivale a “hacia la derecha”.
- 180deg, que equivale “hacia abajo”.

A continuación, se muestran algunos **ejemplos**.

```
#grad {  
    background-image: linear-gradient(to right, red , yellow);  
}
```



```
#grad {  
    background-image: linear-gradient(180deg, red, yellow);  
}
```



```
#grad {  
    background-image: linear-gradient(red, yellow, green);  
}
```





```
#grad {  
  background-image: linear-gradient(to right, rgba(255,0,0,0), rgba(255,0,0,1));  
}
```



```
#grad {  
  background-image: linear-gradient(to bottom right, red, yellow);  
}
```



```
#grad1 {  
  height: 200px;  
  background-image: linear-gradient(red 0%, blue 20%, green 50%,  
  pink 70%, yellow 90%);  
}
```



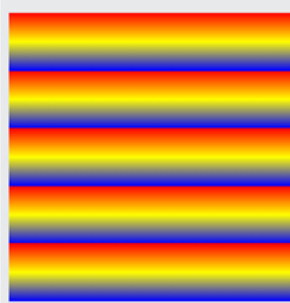
Si queremos repetir el gradiente lineal, podemos usar la propiedad **repeating-linear-gradient()**

Linear Gradient



linear-gradient(red, yellow, blue);

Repeating Linear Gradient



repeating-linear-gradient(red, yellow 10%, blue 20%);

5.2.1.2 Degradado radial ('radial-gradient')

Para crear un degradado radial, se deben también definir al menos dos toques de color o 'color stops'.

Su **sintaxis** es la siguiente:



```
etiqueta html {  
    background-image: radial-gradient(shape, start-color, ..., last-color);  
}
```

```
etiqueta html {  
    background-image: radial-gradient(shape size, start-color, ..., last-color);  
}
```

```
etiqueta html {  
    background-image: radial-gradient(shape, start-color size, ..., last-color size);  
}
```

```
etiqueta html {  
    background-image: radial-gradient(start-color size, ..., last-color);  
}
```

```
etiqueta html {  
    background-image: radial-gradient(shape size at position, start-color, ..., last-color);  
}
```

Por **defecto**, el valor de *shape* es *ellipse*, *size* es *farthest-corner*, y *position* es *center*.

El parámetro de *shape* también puede tomar el valor de *circle*.

El parámetro de *size* define el tamaño del degradado. Puede tomar los siguientes valores:

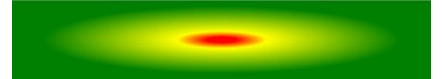
- *closest-side*: la **forma final** del degradado se encuentra con **el lado de la caja más cercano a su centro** (para los círculos) o se encuentra con los lados vertical y horizontal más cercanos al centro (para las elipses).
- *farthest-side*: similar al anterior, excepto que la **forma final** está dimensionada para encontrarse con el **lado de la caja más alejado de su centro** (o los lados vertical y horizontal).



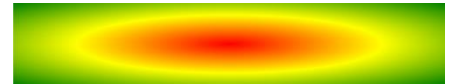
- *closest-corner*: la forma de terminación del degradado tiene un **tamaño** tal que **coincide exactamente con la esquina más cercana** de la caja desde su centro.
- *farthest-corner*: es el valor por defecto. La **forma** final del gradiente tiene un tamaño tal que se encuentra exactamente con la **esquina más lejana de la caja desde su centro**.

A continuación, se muestran algunos **ejemplos**:

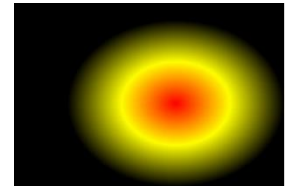
```
#grad {  
  background-image: radial-gradient(red 5%, yellow 15%, green 60%);  
}
```



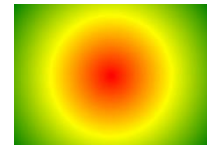
```
#grad {  
  background-image: radial-gradient(red, yellow, green);  
}
```



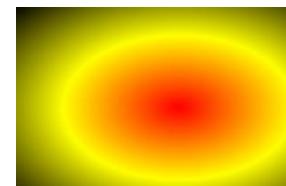
```
#grad1 {  
  height: 150px;  
  width: 220px;  
  background-image: radial-gradient(closest-side at 60% 55%, red,  
  yellow, black);  
}
```



```
#grad {  
  background-image: radial-gradient(circle, red, yellow, green);  
}
```

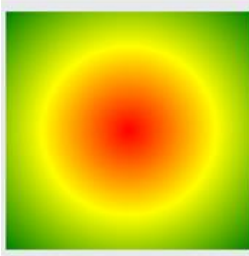


```
#grad4 {  
  height: 150px;  
  width: 220px;  
  background-image: radial-gradient(farthest-corner at 60% 55%, red,  
  yellow, black);  
}
```



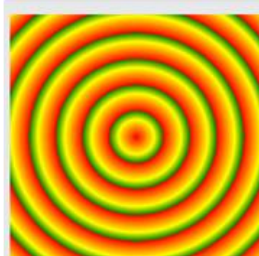
Si queremos repetir el gradiente radial, podemos usar la propiedad **repeating-radial-gradient()**

Radial Gradient



radial-gradient(red, yellow, green);

Repeating Radial Gradient



repeating-radial-gradient(red, yellow 10%, green 15%);

5.2.1.3 Degradado cónico ('conic-gradient')

Un degradado cónico es un degradado con **transiciones de color rotadas alrededor de un punto central**.

Para crear un degradado cónico se debe definir al menos dos colores.

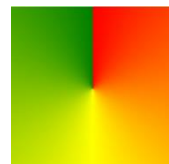
Su **sintaxis** es la siguiente:

```
etiqueta html {
background-image: conic-gradient([from angle] [at position,] color [degree],
color [degree], ...); }
```

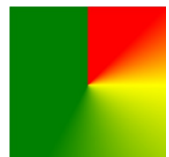
Por **defecto**, el **ángulo** es *0deg* y la **posición** es *center*. Si no se especifica ningún grado, los colores se repartirán por igual alrededor del punto central.

A continuación, se muestran algunos ejemplos:

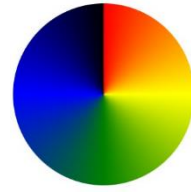
```
#grad1 {
height: 200px;
width: 200px;
background-image: conic-gradient(red, yellow, green);
}
```



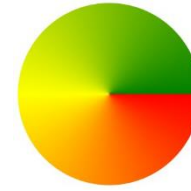
```
#grad1 {
height: 200px;
width: 200px;
background-image: conic-gradient(red 45deg, yellow 90deg, green 210deg);
}
```



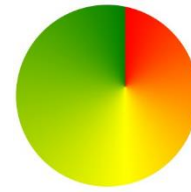
```
#grad1 {
  height: 200px;
  width: 200px;
  background-image: conic-gradient(red, yellow, green, blue, black);
  border-radius: 50%;
}
```



```
#grad1 {
  height: 200px;
  width: 200px;
  background-image: conic-gradient(from 90deg, red, yellow, green);
  border-radius: 50%;
}
```

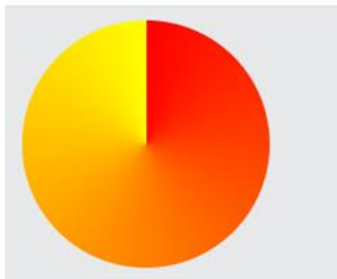


```
#grad1 {
  height: 200px;
  width: 200px;
  background-image: conic-gradient(at 60% 45%, red, yellow, green);
  border-radius: 50%;
}
```



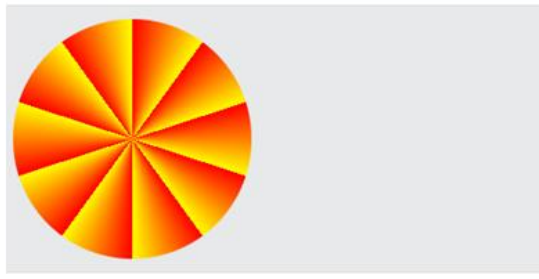
Si queremos repetir el gradiente radial, podemos usar la propiedad **repeating-conic-gradient()**

Conic Gradient



conic-gradient(red, yellow);

Repeating Conic Gradient



repeating-conic-gradient(red 10%, yellow 20%);

5.2.1.4 Generadores de gradientes

Para facilitarnos la tarea de la **creación de gradientes** podemos utilizar un **generador de gradientes online** que nos proporcione el código CSS necesario para nuestro diseño. Algunas herramientas online útiles son las siguientes:

- cssgradient.io
- <https://www.colorzilla.com/es/>

5.3 Repetición y posición

En cuanto a las imágenes, CSS incorpora dos propiedades más relativas a la distribución de la imagen, ‘background-repeat’ y ‘background-position’. La primera permite indicar la **repetición** de una imagen en el fondo y la segunda, la **posición exacta** de la imagen.

5.3.1 background-repeat

La propiedad ‘background-repeat’ permite escoger la dirección de repetición de la imagen. En algunos casos es aconsejable que la imagen se repita en todas las direcciones; por ejemplo, si es demasiado pequeña. Para este fin, se utiliza el valor **repeat**. Si solo se desea que se repita en una dirección, bastará con añadir la dirección de repetición.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    background-repeat: repeat | repeat-x | repeat-y | no-repeat | space  
    | round | initial | inherit ;  
}
```

- **repeat**. La imagen de fondo se repite tanto vertical como horizontalmente. La última imagen se recortará si no cabe. Esta es la opción por defecto.
- **repeat-x**. La imagen de fondo se repite sólo horizontalmente.
- **repeat-y**. La imagen de fondo se repite sólo verticalmente.
- **no-repeat**. La imagen de fondo no se repite. La imagen sólo se mostrará una vez.
- **space**. La imagen de fondo se repite tanto como sea posible sin recortarla. La primera y la última imagen se fijan a ambos lados del elemento, y el espacio en blanco se distribuye uniformemente entre las imágenes.
- **round**. La imagen de fondo se repite y se aplasta o estira para llenar el espacio (sin espacios).

A continuación, se muestran algunos ejemplos:

```
#example1 {  
    border: 2px solid black;  
    padding: 25px;  
    background: url("w3css.gif");  
    background-repeat: repeat;  
}
```





```
#example2 {  
  border: 2px solid black;  
  padding: 25px;  
  background: url("w3css.gif");  
  background-repeat: space;  
}
```



5.3.2 background-position

Además, CSS permite controlar la posición de la imagen dentro del fondo del elemento mediante la propiedad `'background-position'`. La sintaxis de esta propiedad incluye la distancia a cada extremo, teniendo en cuenta que **la posición original de toda imagen es la esquina superior izquierda**.

La **sintaxis** es la siguiente:

```
etiqueta html {  
  background-position: value | inherit | initial ;  
}
```

value puede tomar los siguientes valores:

- `left top | left center | left bottom | right top | right center | right bottom | center top | center center | center bottom`. Si solo se especifica una keyword, por ejemplo `left`, el otro valor será **center por defecto**.
- `x% y%`. El primer valor es la posición horizontal y el segundo la vertical. La esquina superior izquierda es `0% 0%`. La esquina inferior derecha es `100% 100%`. Si sólo especifica un valor, el otro valor será el `50%`. El valor por defecto es `0% 0%`
- `xpos ypos`. El primer valor es la posición horizontal y el segundo la vertical. La esquina superior izquierda es `0 0`. Las unidades pueden ser píxeles (`0px 0px`) o cualquier otra unidad CSS. Si sólo especifica un valor, el otro valor será el `50%`. Se pueden **mezclar % y posiciones**.

Existe otra propiedad relativa a las imágenes especificadas mediante `'background-imagen'` que es `'background-attachment'`. Esta propiedad determina si la imagen estará fija dentro

de la pantalla (**fixed**) o se desplazará junto al bloque contenedor (**scroll**) y se mostrará con una barra de scroll. Este es el valor por defecto.

A continuación, se muestran algunos **ejemplos**:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-image: url('w3css.gif');
  background-repeat: no-repeat;
  background-attachment: fixed;
  background-position: 50px 150px;
}
</style>
</head>
<body>

<h1>The background-position Property</h1>
<p>Here, the background image is positioned 50px
from the left, and 150px down from the top.</p>

</body>
</html>
```

The background-position Property

Here, the background image is positioned 50px from the left, and 150px down from the top.



```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-image: url('w3css.gif');
  background-repeat: no-repeat;
  background-attachment: fixed;
  background-position: right top;
}
</style>
</head>
<body>

<h1>The background-position
Property</h1>
<p>Here, the background image will be
positioned bottom right.</p>

</body>
</html>
```

The background-position Property

Here, the background image will be positioned bottom right.





5.4 background-size

La propiedad **background-size** especifica el tamaño de las imágenes de fondo. Para utilizar esta propiedad, debemos consultar su compatibilidad con los diferentes navegadores web y sus versiones, y la necesidad de utilizar prefijos (-webkit-, -moz-, etc).

Su **sintaxis** es la siguiente:

```
background-size: auto | length | cover | contain;
```

- **auto**. Es el valor por defecto. La imagen de fondo se muestra en su tamaño original.
- **length**. Establece la anchura (width) y la altura (height) de la imagen de fondo. El primer valor establece la anchura, el segundo valor establece la altura. **Si sólo se da un valor, el segundo se establece en "auto"**. Se puede usar unidades de medida o porcentajes.
- **cover**. **Redimensiona** la imagen de fondo para que cubra todo el contenedor, aunque tenga que estirar la imagen o cortar un poco uno de los bordes. Se suele utilizar para **un responsive design**.
- **contain**. **Redimensiona** la imagen de fondo para que la imagen sea totalmente visible.

A continuación, se presenta un ejemplo:



```
<!DOCTYPE html>
<html>
<head>
<style>
#example1 {
  border: 2px solid black;
  padding: 25px;
  background: url(mountain.jpg);
  background-repeat: no-repeat;
  background-size: 100% 100%;
}

#example2 {
  border: 2px solid black;
  padding: 25px;
  background: url(mountain.jpg);
  background-repeat: no-repeat;
  background-size: 75% 50%;
}

#example3 {
  border: 2px solid black;
  padding: 25px;
  background: url(mountain.jpg);
  background-repeat: no-repeat;
  background-size: cover;
}

#example4 {
  border: 2px solid black;
  padding: 25px;
  background: url(mountain.jpg);
  background-repeat: no-repeat;
  background-size: contain;
}

</style>
</head>
<body>

<h2>background-size: 100% 100%;</h2>
<div id="example1">
  <h2>Montains</h2>
  <p>El tamaño de la imagen de fondo se establece en
porcentaje con respecto al elemento padre; 100% de ancho
y 100% de alto.</p>
</div>

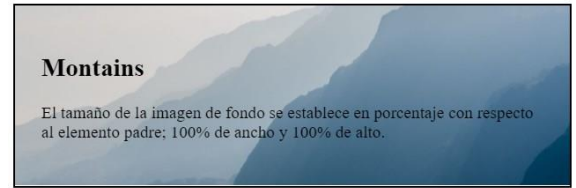
<h2>background-size: 75% 50%;</h2>
<div id="example2">
  <h2>Montains</h2>
  <p>El tamaño de la imagen de fondo se establece en
porcentaje con respecto al elemento padre; 75% width and
50% height.</p>
</div>

<h2>background-size: cover;</h2>
<div id="example3">
  <h2>Montains</h2>
  <p>La imagen de fondo se redimensiona para cubrir todo
el contenedor, aunque tenga que estirar la imagen o
cortar un poco uno de los bordes.</p>
</div>

<h2>background-size: contain;</h2>
<div id="example4">
  <h2>Montains</h2>
  <p>La imagen se redimensiona para que sea
completamente visible.</p>
</div>

</body>
</html>
```

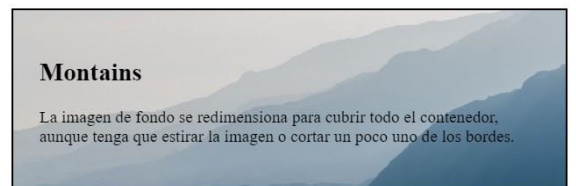
background-size: 100% 100%:



background-size: 75% 50%:



background-size: cover:



background-size: contain:





5.5 Opacity

El estándar de CSS 2.1 no incluye ninguna propiedad para controlar la opacidad de los elementos de la página. Sin embargo, la versión CSS3 incluye una propiedad llamada **opacity**, que permite incluir transparencias en el diseño de la página.

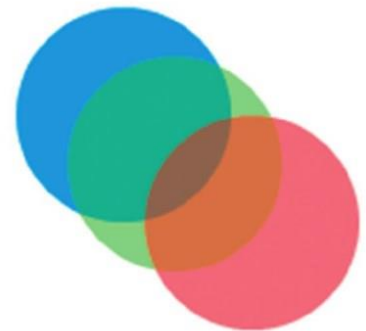
La **sintaxis** es la siguiente:

```
etiqueta html {  
    opacity: valor;  
}
```

El valor de la propiedad '**opacity**' se define mediante un número decimal, cuyo valor puede estar comprendido **entre 0.0 y 1.0**, siendo 0.0 la máxima transparencia, es decir, el elemento es invisible; y 1.0 la máxima opacidad, esto es, el elemento es completamente visible.

En el siguiente **ejemplo** se establece el valor de transparencia al 50%, para los elementos verde y rojo.

```
#verde, #rojo {  
    opacity: 0.5;  
}  
  
#azul {  
    background-color: blue;  
}  
  
#rojo {  
    background-color: red;  
}  
  
#verde {  
    background-color: green;  
}
```





6. Efecto Parallax

Además de “parallax”, esta técnica también es conocida por otros términos en el ámbito del diseño web y la animación. Algunos de los nombres y conceptos relacionados incluyen:

- **Scrolling effect (efecto de desplazamiento).** Se refiere a cualquier tipo de animación o cambio visual que ocurre al desplazarse por la página.
- **Parallax scrolling.** Específicamente se refiere a la técnica donde las capas de contenido se mueven a diferentes velocidades para crear un efecto de profundidad.
- **Depth effect (efecto de profundidad).** Hace referencia a la percepción de tridimensionalidad creada por el movimiento diferencial de elementos.
- **Layered scrolling.** Implica el uso de múltiples capas de contenido que se desplazan a velocidades diferentes.
- **Scroll-based animation (animación basada en desplazamiento).** Incluye cualquier tipo de animación que se activa mediante el desplazamiento del usuario.
- **Fixed background scrolling.** Una variación del efecto parallax donde el fondo se fija y el contenido se desplaza sobre él.

6.1 Pasos para crear un efecto parallax

A continuación, se describen los pasos generales para crear un efecto parallax.

6.1.1 Paso 1: crear los contenedores en HTML

En primer lugar, debemos crear la estructura básica del código HTML. En este caso, creamos un contenedor principal que contendrá el fondo con el efecto parallax y un contenedor dentro del contenedor principal para mostrar el contenido:

- `<div class="parallax">`. Este es el contenedor principal que tendrá la imagen de fondo con el efecto parallax.
- `<div class="content">`. Este contenedor es para el contenido que se encuentra dentro del parallax. En este ejemplo, incluye un encabezado (`<h1>`) y un párrafo (`<p>`).



```
<div class="parallax">
<div class="content">
<h1>Bienvenido a Nuestro Sitio</h1>
<p>Disfruta de este efecto parallax mientras te desplazas.</p>
</div>
```

6.1.2 Paso 2: agregar las propiedades CSS

El siguiente paso es añadir el CSS para crear el efecto parallax. Para hacer un efecto parallax necesitaremos conocer ciertas propiedades para aplicarlas sobre el contenedor principal. En nuestro caso el contenedor con la clase .parallax. Veamos las propiedades aplicar:

- **.parallax**: Aplicamos estilos al contenedor del parallax.
 - **background-image**: URL de la imagen de fondo.
 - **height**: Define la altura del contenedor.
 - **background-attachment**: Fija la imagen de fondo para crear el efecto parallax.
 - **background-position**: Centra la imagen de fondo.
 - **background-repeat**: Evita que la imagen de fondo se repita.
 - **background-size**: Asegura que la imagen cubra todo el contenedor.

```
body, html {
    margin: 0;
    font-family: Arial, sans-serif; }

.parallax {
    background-image: url('tuimagen.jpg');
    height: 600px;
    background-attachment: fixed;
    background-position: center;
    background-repeat: no-repeat;
    background-size: cover; }

.content {
    height: 500px;
    text-align: center;
    padding: 50px; }
```



6.2 Ejemplo sencillo

A continuación, se muestran un de ejemplos de efecto parallax sencillos con fondo fijo.

Ejemplo1

HTML

```
<div class="cta-container">
  <h2 class="cta-text">¡Descubre nuestro producto estrella!</h2>
  <p class="cta-text">Obtén un 20% de descuento en tu primera compra</p>
  <button class="cta-button">Comprar Ahora</button>
</div>
```

CSS

```
*{font-family: Arial;}
.cta-container {
  background-image: url("https://img.freepik.com/vector-gratis/fondo-
monocromatico-blanco-estilo-papel_52683-
66443.jpg?w=900&t=st=1697252118~exp=1697252718~hmac=c828b4d3b79a4f8c77d76e
f1d503fd6b1666bcb1e7b4434ad3dbaffaaadc84ff"); /* Imagen de fondo */
  background-size: cover; /* Cubre completamente el contenedor */
  background-position: center center; /* Centra la imagen */
  background-repeat: no-repeat; /* No se repite la imagen de fondo */
  background-attachment: fixed; /* Fondo fijo en la ventana */
  opacity: 0.9; /* Opacidad del contenedor */
  text-align: center; /* Alineación del contenido al centro */
  padding: 70px; }
.cta-text {
  font-size: 24px; }
.cta-button {
  background-color: #FF5733; /* Color de fondo del botón */
  color: #fff; /* Color del texto del botón en blanco */
  padding: 10px 20px; /* Relleno del botón */
  border: none; /* Sin borde */
  border-radius: 5px; /* Bordes redondeados */
  font-size: 18px; }
.cta-button:hover {
  background-color: #fc2a00; }
```



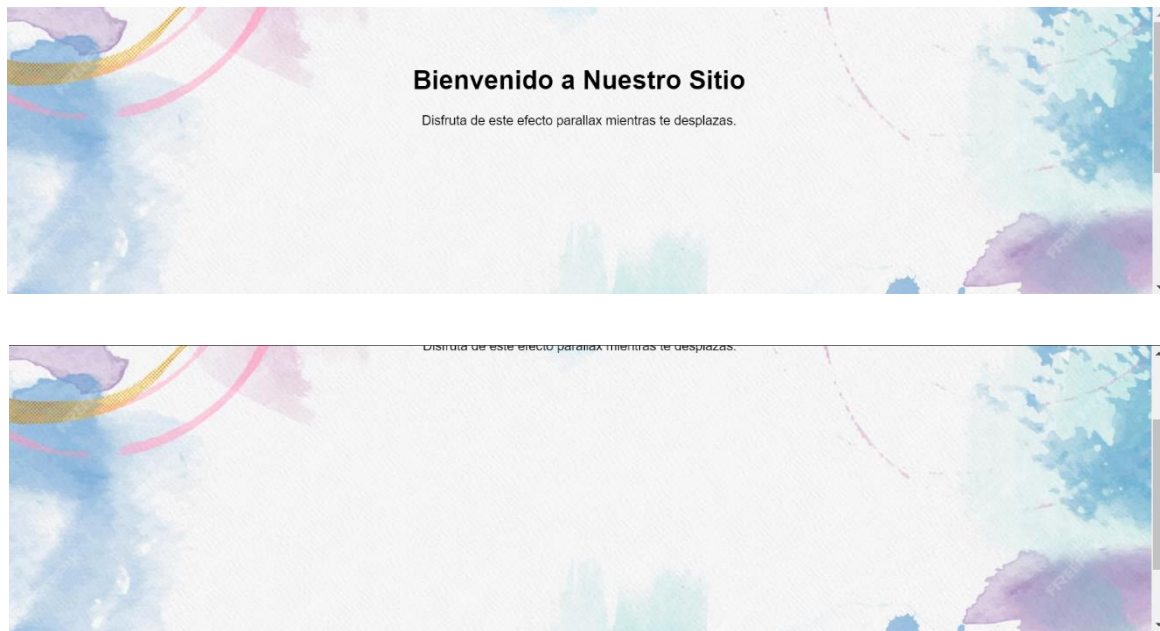

Ejemplo2

HTML

```
<div class="parallax">
  <div class="content">
    <h1>Bienvenido a Nuestro Sitio</h1>
    <p>Disfruta de este efecto parallax mientras te desplazas.</p>
  </div>
```

CSS

```
body, html {
  margin: 0;
  font-family: Arial, sans-serif;
}
.parallax {
  background-image: url('https://img.freepik.com/vector-gratis/fondo-
acuarela-lamina-dorada_52683-
46208.jpg?w=1380&t=st=1718787098~exp=1718787698~hmac=9bb13d994e033cbf04099
6bb966682add623588b6e3f47fc9cd1511ef616bc3a');
  height: 600px;
  background-attachment: fixed;
  background-position: center;
  background-repeat: no-repeat;
  background-size: cover;
}
.content {
  height: 500px;
  text-align: center;
  padding: 50px;
}
```



7. Propiedades de texto

Se denominan propiedades de texto aquellas que **afectan al texto en su conjunto como bloque**. Estas propiedades permiten controlar la alineación del texto, el interlineado, la separación entre palabras, etc.

En la siguiente tabla se muestran **las propiedades de texto** más utilizadas y que vamos a ver en detalle.

Propiedad	Descripción	Valores
<i>text-indent</i>	Desplazamiento de la primera línea del texto	longitud porcentaje
<i>text-align</i>	Alineamiento del texto	left right center justify
<i>text-decoration</i>	Efectos de subrayado, tachado	none underline overline line-through
<i>letter-spacing</i>	Espacio entre caracteres	normal longitud
<i>word-spacing</i>	Espacio entre palabras	normal longitud
<i>text-transform</i>	Transformación a mayúsculas / minúsculas	capitalize uppercase lowercase none
<i>line-height</i>	Tamaño del espacio entre líneas	longitud porcentaje
<i>vertical-align</i>	Alineación vertical	top middle bottom baseline sub super valor

7.1 Interlineado de texto ('line-height')

Esta propiedad indica el tamaño del espacio entre las líneas. Suele encontrarse en forma de **porcentaje**, siendo 100% el interlineado normal. El valor puede ser expresado en porcentaje o en unidades de medida. El uso de un buen interlineado facilita la lectura del texto del sitio web notablemente.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    line-height: valor;  
}
```



Ejemplo:

```
p {  
  line-height: 120%;  
}  
  
p {  
  line-height: 1.2em;  
}
```

Además de las unidades de medida y los porcentajes, la propiedad ‘**line-height**’ puede indicarse por un número sin unidades, que se interpreta como un múltiplo del tamaño normal de la letra.

7.2 Alineado de texto (‘text-align’)

Esta propiedad **establece la alineación del texto**. Los valores definidos para alinear el texto son a la izquierda (**left**), a la derecha (**right**), centrado (**center**) y justificado (**justify**). Si no se indica ningún valor, se utiliza **left por defecto**.

La **sintaxis** es la siguiente:

```
etiqueta html {  
  text-align: left | right | center | justify | initial;  
}
```

7.3 Decoración de texto (‘text-decoration’)

La propiedad ‘**text-decoration**’ añade al texto distintos elementos conocidos: tachado, subrayado, etc. Existe la posibilidad de poner **varios valores al mismo tiempo** en esta propiedad.

El **valor por defecto** es **none**, y, aunque cabe esperar que no se vaya a utilizar nunca, puesto que si no deseamos ninguna decoración basta con indicarlo, existen algunos casos, como **los enlaces** en los que aparecen subrayados por defecto; si se deseara eliminar este efecto, tendría que **utilizarse none**.

La **sintaxis** es la siguiente (todas las propiedades en la misma regla):

```
etiqueta html {
  text-decoration: text-decoration-line text-decoration-color text-
  decoration-style text-decoration-thickness | initial | inherit;
}
```

Diagram illustrating the syntax of the `text-decoration` property, showing its components:

- `text-decoration-line`: none | underline | overline | line-through | blink | initial;
- `text-decoration-color`: color | rgb() | hsl () | initial;
- `text-decoration-style`: solid | double | dotted | dashed | wavy | initial;
- `text-decoration-thickness`: auto | from-font | length/percentage | initial | inherit;

Como se puede ver en la sintaxis, **esta propiedad está formada por otros tipos**:

- 1) En primer lugar, '`text-decoration-line`' indica el lugar exacto en el que se va a añadir la línea sobre el texto (superior, inferior o media).
- 2) En segundo lugar, '`text-decoration-color`', el cual define el color.
- 3) La propiedad que define el estilo de dicha línea, '`text-decoration-style`'.
- 4) Por último, '`text-decoration-thickness`' indica el grosor de la línea.

text-decoration-line		
<i>Underline</i>	Subraya el texto con una línea en su parte inferior.	Ejemplo
<i>Overline</i>	Incorpora una línea en la parte superior del texto.	Ejemplo
<i>Line-Through</i>	Incorpora una línea en la parte media del texto, lo tacha.	Ejemplo
text-decoration-color		
Define el color usando una palabra clave o codificación en hexadecimal	<pre>div { text-decoration-color: green; } div { text-decoration-color: #008000; }</pre>	
Define el color usando el sistema RGB	<pre>div { text-decoration-color: rgb(255,0,0); }</pre>	
Define el color usando el sistema HSL	<pre>div { text-decoration-color: hsl(120,100%,50%); }</pre>	



text-decoration-style		
<i>solid</i>	Valor por defecto. Muestra la línea sin ningún estilo adicional.	
<i>double</i>	Muestra dos líneas.	
<i>dotted</i>	Muestra una línea punteada.	
<i>dashed</i>	Muestra una línea discontinua.	

Ejemplo 1:

```
<!DOCTYPE html>
<html>
<head>
<style>
  h1 {
    text-decoration: underline;
    text-decoration-thickness: auto;
  }

  h2 {
    text-decoration: underline;
    text-decoration-thickness: 5px;
  }
  h3 {
    text-decoration: underline;
    text-decoration-thickness: 50%;
  }
  |
  /* Use shorthand property */
  h4 {
    text-decoration: underline solid red 50%;
  }
</style>
</head>
<body>

<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
<h4>This is heading 4</h4>

</body>
</html>
```

This is heading 1

This is heading 2

This is heading 3

This is heading 4



Ejemplo 2:

```
<!DOCTYPE html>
<html>
<head>
<style>
h1 {
  text-decoration: underline overline dotted red;
}

h2 {
  text-decoration: underline wavy blue 5px;
}
</style>
</head>
<body>

<h1>This is heading 1</h1>
<h2>This is heading 2</h2>

</body>
</html>
```

This is heading 1

This is heading 2

7.4 Alineación vertical ('vertical-align')

La propiedad `vertical-align` puede utilizarse en **dos contextos**:

- 1) Para alinear verticalmente la caja de un elemento de **nivel inline dentro de su caja de línea contenedora**. Por ejemplo, puede utilizarse para posicionar verticalmente una imagen en una línea de texto.
- 2) Para **alinear verticalmente el contenido de una celda en una tabla**.

Su **sintaxis** es la siguiente:

```
etiqueta html {
  vertical-align: baseline | length | sub | super | top | text-top
  | middle | bottom | text-bottom | initial | inherit;
}
```

La alineación vertical **sólo se aplica a los elementos inline, inline-block y table-cell**:



Se puede encontrar más información en: https://www.w3schools.com/cssref/pr_pos_vertical-align.php

Ejemplo de alineación vertical de una imagen en una línea de texto:

```
<!DOCTYPE html>
<html>
<head>
<style>
img.a {
  vertical-align: baseline;
}

img.b {
  vertical-align: text-top;
}

img.c {
  vertical-align: text-bottom;
}

img.d {
  vertical-align: sub;
}

img.e {
  vertical-align: super;
}
</style>
</head>
<body>

<h1>The vertical-align Property</h1>

<h2>vertical-align: baseline (default):</h2>
<p>An 
image with a default alignment.</p>

<h2>vertical-align: text-top:</h2>
<p>An 
image with a text-top alignment.</p>

<h2>vertical-align: text-bottom:</h2>
<p>An 
image with a text-bottom alignment.</p>

<h2>vertical-align: sub:</h2>
<p>An 
image with a sub alignment.</p>

<h2>vertical-align: sup:</h2>
<p>An 
image with a super alignment.</p>

</body>
</html>
```

The vertical-align Property

vertical-align: baseline (default):

An  image with a default alignment.

vertical-align: text-top:

An  image with a text-top alignment.

vertical-align: text-bottom:

An  image with a text-bottom alignment.

vertical-align: sub:

An  image with a sub alignment.

vertical-align: sup:

An  image with a super alignment.

Ejemplo de alineación vertical del contenido de una celda dentro de una tabla:

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      td{height:100px; border: 1px solid grey;}
      .a { vertical-align: top; }
      .b { vertical-align: middle; }
      .c { vertical-align: bottom; }
    </style>
  </head>
  <body>
    <h3>Propiedad vertical-align</h3>
    <table>
      <tr>
        <td class="a">vertical-align: top</td>
        <td class="b">vertical-align: middle</td>
        <td class="c">vertical-align: bottom</td>
      </tr>
    </table>
  </body>
</html>
```

Propiedad vertical-align

vertical-align: top	vertical-align: middle	vertical-align: bottom
---------------------	------------------------	------------------------

7.5 Transformación del texto (“text-transform”)

La propiedad CSS “text-transform” controla las mayúsculas del texto. Su **sintaxis** es la siguiente:

```
etiqueta html {
  text-transform: none | capitalize | uppercase | lowercase;
}
```

- **capitalize**: convierte la primera letra de cada palabra en mayúscula.
- **uppercase**: convierte todo el texto en mayúscula.
- **lowercase**: convierte todo el texto en minúscula.



Ejemplo:

```
<!DOCTYPE html>
<html>
<head>
<style>
  div.a {
    text-transform: uppercase;
  }
  div.b {
    text-transform: lowercase;
  }
  div.c {
    text-transform: capitalize;
  }
</style>
</head>
<body>
  <h1>The text-transform Property</h1>

  <h2>text-transform: uppercase:</h2>
  <div class="a">Lorem ipsum dolor sit amet, consectetur adipiscing elit.</div>

  <h2>text-transform: lowercase:</h2>
  <div class="b">Lorem ipsum dolor sit amet, consectetur adipiscing elit.</div>

  <h2>text-transform: capitalize:</h2>
  <div class="c">Lorem ipsum dolor sit amet, consectetur adipiscing elit.</div>

</body>
</html>
```

The text-transform Property

text-transform: uppercase:

LOREM IPSUM DOLOR SIT AMET, CONSECTETUR ADIPISCING ELIT.

text-transform: lowercase:

lorem ipsum dolor sit amet, consectetur adipiscing elit.

text-transform: capitalize:

Lorem Ipsum Dolor Sit Amet, Consectetur Adipiscing Elit.

7.6 Letter spacing

Esta propiedad CSS aumenta o disminuye el **espacio existente entre los caracteres** de un texto, es decir, especifica el espacio que hay entre las letras de un texto. Puede tomar valores negativos.

Su sintaxis es la siguiente:

```
etiqueta html {
  letter-spacing: normal | length;
}
```

Ejemplo:

```
<!DOCTYPE html>
<html>
<head>
<style>
  h1 {
    letter-spacing: 3px;
  }
  h2 {
    letter-spacing: -1px;
  }
  h3 {
    letter-spacing: normal;
  }
</style>
</head>
<body>
  <h1>Espaciado entre letras de 3px</h1>
  <h2>Espaciado entre letras de 2px</h2>
  <h3>Espaciado entre letras normal</h3>

</body>
</html>
```

Espaciado entre letras de 3px

Espaciado entre letras de 2px

Espaciado entre letras normal



7.7 Word spacing

Esta propiedad CSS aumenta o disminuye el **espacio en blanco entre las palabras**, es decir, especifica el espacio que hay entre palabras.

Su sintaxis es la siguiente:

```
etiqueta html {  
    word-spacing: normal | length;  
}
```

- **normal**: es el valor por defecto y equivale a un espaciado entre palabras de 0.25 em.
- **length**: define el espacio entre palabras en px, pt, cm, em, etc. No admite valores negativos.

Ejemplo:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
    p.a {  
        word-spacing: normal;  
    }  
    p.b {  
        word-spacing: 30px;  
    }  
    p.c {  
        word-spacing: 1cm;  
    }  
</style>  
</head>  
<body>  
  
    <h1>The word-spacing Property</h1>  
  
    <h2>word-spacing: normal:</h2>  
    <p class="a">This is some text. This is some text.</p>  
  
    <h2>word-spacing: 30px:</h2>  
    <p class="b">This is some text. This is some text.</p>  
  
    <h2>word-spacing: 1cm:</h2>  
    <p class="c">This is some text. This is some text.</p>  
  
</body>  
</html>
```

The word-spacing Property

word-spacing: normal:

This is some text. This is some text.

word-spacing: 30px:

This is some text. This is some text.

word-spacing: 1cm:

This is some text. This is some text.



7.8 Tabulado del texto ('text-indent')

La propiedad **'text-indent'** permite tabular la primera línea de cada párrafo creando una sangría de primera línea. El valor puede expresarse en porcentaje o en unidades métricas.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    text-indent: <medida> | <porcentaje> ;  
}
```

Ejemplo:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
    div.a {  
        text-indent: 50px;  
    }  
    div.b {  
        text-indent: -2em;  
    }  
    div.c {  
        text-indent: 30%;  
    }  
</style>  
</head>  
<body>  
    <h1>The text-indent Property</h1>  
    <h2>text-indent: 50px:</h2>  
    <div class="a">  
        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam  
        semper diam at erat pulvinar, at pulvinar felis blandit. </p>  
    </div>  
    <h2>text-indent: -2em:</h2>  
    <div class="b">  
        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.  
        Etiam semper diam at erat pulvinar, at pulvinar felis blandit. </p>  
    </div>  
    <h2>text-indent: 30%:</h2>  
    <div class="c">  
        <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.  
        Etiam semper diam at erat pulvinar, at pulvinar felis blandit. </p>  
    </div>  
</body>  
</html>
```

The text-indent Property

text-indent: 50px:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam semper diam at erat pulvinar, at pulvinar felis blandit. Vestibulum volutpat tellus diam, consequat gravida libero rhoncus ut.

text-indent: -2em:

em ipsum dolor sit amet, consectetur adipiscing elit. Etiam semper diam at erat pulvinar, at pulvinar felis blandit. Vestibulum volutpat tellus diam, consequat gravida libero rhoncus ut.

text-indent: 30%:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam semper diam at erat pulvinar, at pulvinar felis blandit. Vestibulum volutpat tellus diam, consequat gravida libero rhoncus ut.

7.9 Sombra del texto ('text-shadow')

La propiedad **text-shadow** añade **sombra** al texto.

Su **sintaxis** es la siguiente:

```
etiqueta html {  
    text-shadow: h-shadow v-shadow blur-radius color|none;  
}
```



- ***h-shadow***. Requerido. Es la posición de la sombra horizontal. Se permiten valores negativos.
- ***v-shadow***. Requerido. Es la posición de la sombra vertical. Se permiten valores negativos.
- ***blur-radius***. Opcional. Es el radio de desenfoque o difuminado. El valor por defecto es 0.
- ***color***. Opcional. Es el color de la sombra.
- ***none***. Es el valor por defecto. Sin sombra.

IMPORTANTE:

- Para aplicar sombras **arriba y a la izquierda** se deben utilizar **valores negativos**.
- Si queremos usar **múltiples sombras** al mismo elemento, basta con **separar cada sombra con una coma (,)** y respetar la estructura.
- Para este tipo de propiedades debemos incluir los **prefijos para navegadores**.



A continuación, se muestra un ejemplo:

```
<!DOCTYPE html>
<html>
<head>
<style>
h1 {
  text-shadow: 2px 2px red, 4px 4px #FF0000, 8px
8px yellow;
}
</style>
</head>
<body>

<h1>The text-shadow Property</h1>

</body>
</html>
```

The text-shadow Property

7.10 Sombra a elementos caja ('box-shadow')

La propiedad **box-shadow** añade **sombra** a los elementos caja.

Su **sintaxis** es la siguiente:

```
etiqueta html {  
  box-shadow: none | h-offset v-offset blur spread color inset;  
}
```

- **h-offset**. Requerido. Es la posición de la sombra horizontal. Se permiten valores negativos.
- **v-offset**. Requerido. Es la posición de la sombra vertical. Se permiten valores negativos.
- **blur**. Opcional. Es el radio de desenfoque o difuminado.
- **spread**. Opcional. Es el tamaño de la sombra hacia los lados.
- **color**. Opcional. Es el color de la sombra.
- **inset**. Opcional. Posiciona la sombra dentro del marco.

A continuación, se puede ver un **ejemplo**.

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
#example1 {  
  border: 1px solid;  
  padding: 10px;  
  box-shadow: 5px 5px blue, 10px 10px red, 15px 15px green;  
  margin: 20px;  
}  
  
#example2 {  
  border: 1px solid;  
  padding: 10px;  
  box-shadow: 5px 5px 8px blue, 10px 10px 8px red, 15px 15px 8px green;  
  margin: 20px;  
}  
</style>  
</head>  
<body>  
  
<h1>Multiple Shadows</h1>  
  
<div id="example1">  
  <h2>Multiple shadows</h2>  
  <p>box-shadow: 5px 5px blue, 10px 10px red, 15px 15px green:</p>  
</div>  
<br>  
  
<div id="example2">  
  <h2>Multiple shadows with blur effect</h2>  
  <p>box-shadow: 5px 5px 8px blue, 10px 10px 8px red, 15px 15px 8px green:</p>  
</div>  
  
</body>  
</html>
```

Multiple Shadows

Multiple shadows

box-shadow: 5px 5px blue, 10px 10px red, 15px 15px green;

Multiple shadows with blur effect

box-shadow: 5px 5px 8px blue, 10px 10px 8px red, 15px 15px 8px green;



7.10.1 Generadores de sombra online

Para facilitarnos la tarea de la creación de sombras podemos utilizar un generador de sombras tanto para los textos como para las cajas o contenedores. Algunas herramientas online útiles son las siguientes:

- **Generador de sombras en textos:**
 - <https://html-css-js.com/css/generator/text-shadow/>
 - <https://cssgenerator.org/text-shadow-css-generator.html>

- **Generador de sombras en cajas o contenedores:**
 - <https://html-css-js.com/css/generator/box-shadow/>
 - <https://cssgenerator.org/box-shadow-css-generator.html>



8. Propiedades de Fuente

La **elección de una fuente adecuada** para una página web es importante, así como las **propiedades** relacionadas con esta, puesto que el conjunto final permite a los usuarios interpretar de una manera u otra el contenido del sitio web, por ejemplo, marcar los elementos más importantes y que mayor grado e información aportan al usuario final.

En la siguiente tabla se muestran **las propiedades de las fuentes** más destacadas.

Propiedad	Descripción	Valores
font-family	Familias de fuentes	nombre-familia *
font-style	Estilo de la fuente	normal italic oblique
font-variant	Convierte a mayúsculas manteniendo todas las letras en un tamaño inferior a la primera	normal small-caps
font-weight	Anchura de los caracteres. Normal = 400, Negrita = 700	normal bold bolder lighter 100 200 300 400 500 600 700 800 900
font-size	Tamaño de la fuente	xx-small x-small small medium large x-large xx-large larger smaller longitud porcentaje

8.1 Fuente de letra ('font-family')

Esta propiedad establece la **fuentes de la letra utilizada** en el documento estructurado HTML. Como se vio en la primera unidad, existen algunas tipografías cuyo uso es más recomendable, puesto que se trata de fuentes genéricas disponibles en la mayoría de los navegadores; estas son *Serif*, *Sans-Serif*, *Cursive*, *Monospace* o *Fantasy*. Se podría usar cualquier otra fuente, pero debe tenerse en cuenta que, si el navegador no la reconoce, utilizará otra distinta. Para evitar resultados indeseados se recomienda añadir varios tipos de fuentes.

Su **sintaxis** es la siguiente:

```
etiqueta html {  
    font-family: (nombre_familia | familia_genérica),  
                (nombre_familia | familia_genérica)*;  
}
```



Si el navegador no dispone de la primera tipografía indicada, irá probando con el resto hasta encontrar un tipo válido. Por esta razón, se aconseja añadir siempre como última opción una de las fuentes genéricas nombradas, puesto que estas están disponibles en todos los navegadores.

Debe tenerse en cuenta que, **si el nombre de la familia está formado por más de una palabra, se escribe entre comillas**, por ejemplo, “Times New Roman”. Por el contrario, si utilizamos tipografías tales como Courier, no será necesario entrecomillarlo.

Ejemplo:

```
p {  
  font-family: serif;  
}  
  
p {  
  font-family: Arial, Helvetica, sans-serif;  
}  
  
p {  
  font-family: “Times New Roman”, Palatino, times, serif;  
}
```

8.2 Tamaño de letra ('font-size')

La propiedad 'font-size' permite establecer el **tamaño del tipo de letra** del elemento HTML. Se puede expresar el tamaño de varias maneras: tamaño absoluto, tamaño relativo, distancia o porcentaje; generalmente, se aconseja utilizar unidades relativas.

La **sintaxis** es la siguiente:

```
etiqueta html {  
  font-size: valor;  
}
```




- Tamaño **absoluto**:

```
etiqueta html {  
    font-size: xx-small | x-small | small | medium | large |  
              x-large | xx-large;  
}
```

- Tamaño **relativo**:

```
etiqueta html {  
    font-size: smaller | larger;  
}
```

- **Numérico**:

```
etiqueta html {  
    font-size: <longitud> | <porcentaje>;  
}
```

Atributos de tamaño absoluto y relativo

Tamaño absoluto Dimensiones absolutas. Son las indicados por el diseñador y no están en función del resto de elementos, como ocurre con las dimensiones relativas. Se aplica <i>medium</i> por defecto. (14 px.)	• <i>xx-small</i> • <i>x-small</i> • <i>small</i> • <i>medium</i> • <i>large</i> • <i>x-large</i> • <i>xx-large</i>	<pre>p { font-size: x-large; }</pre>
Tamaño relativo Estos tamaños se toman con respecto al elemento "padre", y toman el tamaño siguiente o anterior en la lista de tamaños absolutos visto en la fila anterior.	• <i>larger</i> • <i>smaller</i>	<pre>ul { font-size: smaller; }</pre>
Valor numérico con unidades	Se indica una unidad positiva de longitud (pt,px).	<pre>p { font-size: 16pt; }</pre>
Valor numérico con porcentajes	Se indica un porcentaje del tamaño con respecto al elemento "padre".	<pre>p { font-size: 80%; }</pre>

A continuación, se muestra un **ejemplo**:



```
<!DOCTYPE html>
<html>
<head>
<style>
  div.a {
    font-size: 15px;
  }
  div.b {
    font-size: large;
  }
  div.c {
    font-size: 150%;
  }
  p.d {
    font-size: larger;
  }
</style>
</head>
<body>
  <h1>The font-size Property</h1>

  <div class="a">Es un texto con tamaño 15 px.</div>

  <div class="b">Es un texto con tamaño large.</div>

  <div class="c">Es un texto con tamaño 150%.
    <p class="d"> Parrafo con letra mayor que su
padre </p>
  </div>

</body>
</html>
```

The font-size Property

Es un texto con tamaño 15 px.

Es un texto con tamaño large.

Es un texto con tamaño 150%.

Parrafo con letra mayor que su padre

8.3 Estilo de letra ('font-style')

Esta tercera propiedad relativa a la fuente de la letra se encarga de **modificar la inclinación** de las palabras. En concreto, pueden encontrarse dos tipos de letra inclinada: por un lado, la **itálica** (más conocida como cursiva), y por otro, las **oblicuas**.

La **sintaxis** es la siguiente:

```
etiqueta html {
  font-style: normal | italic | oblique;
}
```



Resumen de estilos de texto

Normal	Normal	<pre>p { font-family: sans-serif; font-size: 200%; font-style: normal; }</pre>
Cursiva	<i>Italic</i>	<pre>p { font-family: sans-serif; font-size: 50%; font-style: italic; }</pre>
Oblicua	<i>Oblique</i>	<pre>p { font-family: sans-serif; font-size: 10%; font-style: oblique; }</pre>

Ejemplo:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
  p.a {  
    font-style: normal;  
  }  
  
  p.b {  
    font-style: italic;  
  }  
  
  p.c {  
    font-style: oblique;  
  }  
</style>  
</head>  
<body>  
  <h1>The font-style Property</h1>  
  
  <p class="a">This is a paragraph, normal.</p>  
  <p class="b">This is a paragraph, italic.</p>  
  <p class="c">This is a paragraph, oblique.</p>  
  
</body>  
</html>
```

The font-style Property

This is a paragraph, normal.

This is a paragraph, italic.

This is a paragraph, oblique.

8.4 Grosor de letra ('font-weight')

Esta propiedad modifica el **grosor** de las letras. La aplicación de esta propiedad se basa en la utilización de un conjunto de valores numéricos ya determinados, desde la opción más estrecha hasta la más gruesa (100, 200, 300, 400, 500, 600, 700, 800, y 900). Ahora bien,



también es posible utilizar las palabras *normal* y *bold*, correspondientes al valor **400**, el grosor usual utilizado en los trazos, y **700**, respectivamente.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    font-weight: valor;  
}
```

Al igual que ocurre con la propiedad 'font-size', esta propiedad incorpora dos valores que permiten reducir o aumentar el grosor a un valor inferior o superior, respectivamente, de la lista de valores posibles. Estos son: *lighter* y *bolder*.

En función del navegador y del tipo de fuente escogido, la diferencia entre grosores es más o menos perceptible.

Resumen grosor de letra ('font-weight')

Normal	Equivale a un grosor de 400.
Bold	Equivale a un grosor de 700.
Lighter	Disminuye al valor inmediatamente inferior al del elemento "padre".
Bolder	Aumenta al valor inmediatamente superior al del elemento "padre".

Ejemplo:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
    p.normal {  
        font-weight: normal;  
    }  
    p.thick {  
        font-weight: bold;  
    }  
    p.lighter {  
        font-weight: lighter;  
    }  
    p.thick2 {  
        font-weight: 900;  
    }  
</style>  
</head>  
<body>  
    <h1>The font-weight Property</h1>  
    <p class="normal">Parrafo con grosor normal.</p>  
    <p class="thick2">Parrafo con grosor de 900.</p>  
    <p class="thick">Parrafo con grosor bold.  
        <p class="lighter">Parrafo con un grosor mas fino que su padre p</p>  
    </p>  
</body>  
</html>
```

The font-weight Property

Parrafo con grosor normal.

Parrafo con grosor de 900.

Parrafo con grosor bold.

Parrafo con un grosor mas fino que su padre p



8.5 Variante de fuente ('font-variant')

La propiedad **font-variant** especifica si un texto debe mostrarse o no en minúsculas, de manera que **convierte a mayúsculas** manteniendo todas las letras en un tamaño inferior a la primera.

Su **sintaxis** es la siguiente:

```
etiqueta html {  
    font-variant: normal|small-caps|initial|inherit;  
}
```

A continuación, se muestra un **ejemplo**:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
p.normal {  
    font-variant: normal;  
}  
  
p.small {  
    font-variant: small-caps;  
}  
</style>  
</head>  
<body>  
<h1>The font-variant Property</h1>  
  
<p class="normal">My name is Hege Refsnes.</p>  
<p class="small">My name is Hege Refsnes.</p>  
  
</body>  
</html>
```

The font-variant Property

My name is Hege Refsnes.

MY NAME IS HEGE REFSNES.

8.6 La propiedad fundamental 'font'

La propiedad principal para definir el tipo de fuente, tamaño y posición, entre otras es la propiedad **'font'**, que se considera una propiedad combinada, puesto que permite definir las siguientes propiedades individuales: **'font-style'**, **'font-variant'**, **'font-weight'**, **'font-size'**, **'line-height'** y **'font-family'**.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    font : font-style, font-variant, font-weight, font-size/line-  
           height, font-family;  
}
```



Es muy importante que **las propiedades aparezcan en el orden indicado**. Si no se hace así, los navegadores no las interpretarán y no darán el formato indicado.

- Siempre deben aparecer las propiedades **'font-size'** y **'font-family'** al utilizar la propiedad **'font'**.
- Si se escribe la propiedad **'line-height'** debe aparecer separada de **'font-size'** por una barra inclinada (/).

Ejemplo:

```
<!DOCTYPE html>
<html>
<head>
<style>
  p.a {
    font: 15px Arial, sans-serif;
  }

  p.b {
    font: italic small-caps bold 12px/30px Georgia, serif;
  }
</style>
</head>
<body>

  <h1>The font Property</h1>

  <p class="a">This is a paragraph. The font size is set to 15
pixels, and the font family is Arial.</p>

  <p class="b">This is a paragraph. The font is set to italic and
bold, with small-caps (all lowercase letters are converted to
uppercase). The font size is set to 12 pixels, the line height is
set to 30 pixels, and the font family is Georgia.</p>

</body>
</html>
```

The font Property

This is a paragraph. The font size is set to 15 pixels, and the font family is Arial.

THIS IS A PARAGRAPH. THE FONT IS SET TO ITALIC AND BOLD, WITH SMALL-CAPS (ALL LOWERCASE LETTERS ARE CONVERTED TO UPPERCASE). THE FONT SIZE IS SET TO 12 PIXELS, THE LINE HEIGHT IS SET TO 30 PIXELS, AND THE FONT FAMILY IS GEORGIA.

Las propiedades **'font'** admite los valores **caption**, **icon**, **menu**, **message-box**, **small-caption** y **status-bar**, que hacen referencia a los tipos de letra que utiliza el sistema operativo para esos elementos.

Palabra clave	Elementos del Sistema Operativo a los que hace referencia
caption	Controles y campos de formulario (botones, listas desplegables, etc)
icon	Texto mostrado debajo de cada icono
menu	Menús desplegables y otros tipos de menús
message-box	Cuadros de diálogo (ventanas con mensajes de error, información, advertencia, etc.)
small-caption	Controles y campos de formulario más pequeños
status-bar	Barra de estado de las ventanas



Ejemplo:

```
<!DOCTYPE html>
<html>
<body>

<h1>The font Property</h1>

<p style="font:caption">The font used in captioned controls.</p>
<p style="font:icon">The font used in icon labels.</p>
<p style="font:menu">The font used in dropdown menus.</p>
<p style="font:message-box">The font used in dialog boxes.</p>
<p style="font:small-caption">A smaller version of the caption font.</p>
<p style="font:status-bar">The font used in the status bar.</p>
<p><b>Note:</b> The result of the font keywords is browser dependant.
</p>

</body>
</html>
```

The font Property

The font used in captioned controls.

The font used in icon labels.

The font used in dropdown menus.

The font used in dialog boxes.

A smaller version of the caption font.

The font used in the status bar.

Note: The result of the font keywords is browser dependant.

8.7 Fuentes externas

Utilizar las **fuentes del sistema** tiene la **desventaja** de que dichas fuentes dependen del navegador web y del sistema operativo del usuario. Por ejemplo, las fuentes que están instaladas en sistemas operativos Windows, como pueden ser la *Times new Roman*, *Tahoma*, *Arial*, etc, se ven correctamente en navegadores con dicho sistema operativo, pero no ocurre lo mismo en dispositivos con Linux o Mac.

Es por ello que si queremos asegurarnos de que todos los usuarios verán la misma tipografía en nuestra página web independientemente del dispositivo que utilicen, o queremos usar una tipografía especial, debemos utilizar fuentes externas.

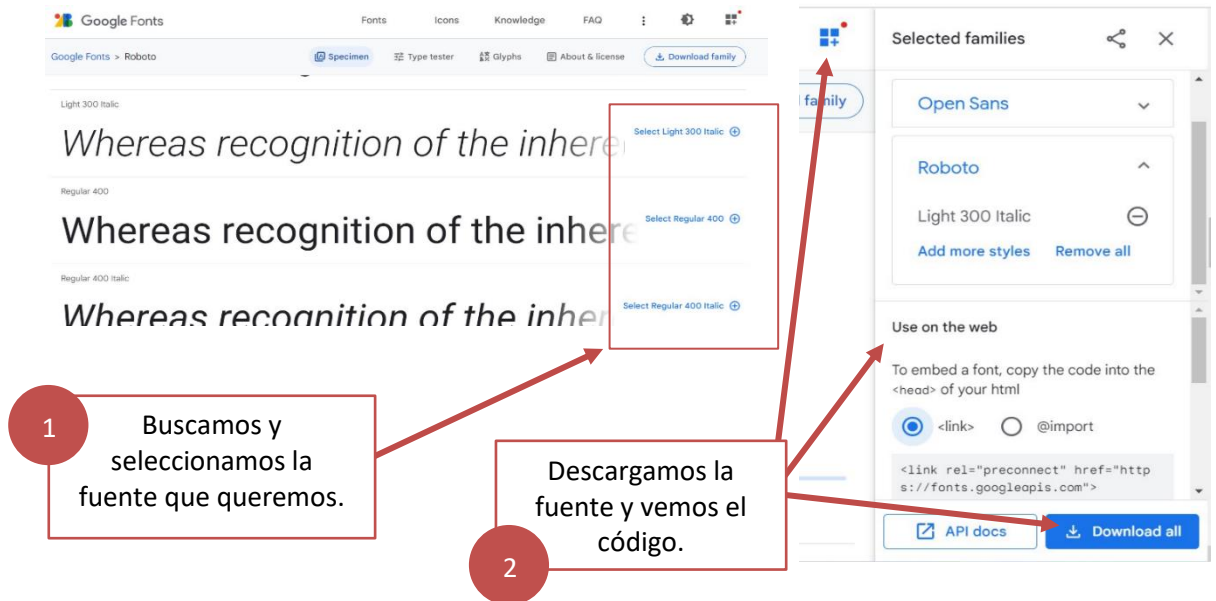
Un catálogo de fuentes muy utilizado es el de [Google Fonts](#). Es bastante cómodo, ya que **Google** nos proporciona un código por cada fuente que tan solo tenemos que copiar y pegar en nuestro proyecto para disponer de la fuente que deseemos.

Tenemos **dos formas** de agregar las fuentes externas a nuestro proyecto:

- 2) A **nivel interno**, descargando la fuente, y mediante el uso de **@font-face**.

Una buena forma de trabajar con las fuentes en local es crear un directorio llamado **fonts**, en el mismo directorio donde tengamos nuestra carpeta **css**, en donde se alojarán los archivos de las fuentes que se vayan a utilizar en el proyecto.

A continuación, se muestra cómo **descargar** las fuentes desde Google Fonts y cómo obtener el **código** por cada fuente:



Para utilizar *@font-face* correctamente debemos definir un nombre para la fuente (por ejemplo, Roboto), y a continuación indicar dónde se encuentra ésta.

Ejemplo:

- 1) En primer lugar, incluimos la fuente Roboto-Regular que se encuentra en la carpeta /fonts.

```
@font-face {  
  font-family: 'Roboto';  
  src: url('../fonts/Roboto/Roboto-Regular.ttf');  
}
```

- 2) Luego, podemos declarar una clase en nuestra hoja de estilos como la que sigue para utilizarla:

```
.roboto {  
  font-family: 'Roboto', sans-serif;  
}
```

- 3) A **nivel externo**, cuando no queremos tener una fuente en local, por ejemplo, por cuestiones de copyright. Esta opción es mucho más sencilla y se puede utilizar si no tenemos reparos para usar recursos externos en nuestra página web. Se puede hacer de dos maneras:



- Incluir fuente con `<link>` a través del HTML. El código nos lo proporciona Google Fonts.

Ejemplo: Si queremos utilizar la fuente Roboto de Google Fonts, debemos seguir los siguientes pasos:

- 1) Agregamos el siguiente código que nos proporciona Google Fonts, en la cabecera o `<head>` de nuestra hoja *HTML*.

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Roboto:ital,wght@1,300&display=swap" rel="stylesheet">
```

- 2) Luego, podemos declarar una clase en nuestra hoja de estilos como la que sigue para utilizarla (según nos indique Google):

```
.roboto {
  font-family: 'Roboto', sans-serif;
}
```

- Incluir fuente con `@import` o a través del CSS. El código nos lo proporciona Google Fonts. El proceso es similar al anterior.

Ejemplo: Si queremos utilizar la fuente Roboto de Google Fonts, debemos seguir los siguientes pasos:

- 1) Agregamos en nuestra hoja de estilos el código que nos indique Google, al principio del documento:

```
@import url('https://fonts.googleapis.com/css2?family=Roboto:ital,wght@1,300&display=swap');
```

- 2) Luego, podemos declarar una clase en nuestra hoja de estilos como la que sigue para utilizarla (según nos indique Google):

```
.roboto {
  font-family: 'Roboto', sans-serif;
}
```



8.7.1 Fuentes web online

Nombre	URL
Font Squirrel	fontsquirrel.com
Da Font	dafont.com
Google Fonts	google.com/fonts

8.7.2 Convertidores de fuentes online

Nombre	URL
Font Squirrel converter	fontsquirrel.com/tools/webfont-generator
Online Font Converter	onlinefontconverter.com
Files conversion	files-conversion.com/font-converter.php
Font converter	fontconverter.org



9. Hyphens CSS

¿Te ha pasado que al diseñar una página web te encuentras con el **problema de que las palabras no caben correctamente en las líneas de texto** y necesitas usar el guion para dividirlos de forma automática? Este inconveniente puede hacer que tus diseños se vean desordenados y poco profesionales. Pero ¡no te preocupes! Existe una **solución simple y efectiva: la propiedad hyphens CSS**.

La propiedad **hyphens** nos **permite controlar la división de palabras al final de una línea de texto, mejorando la legibilidad y estética del contenido**.

9.1 Sintaxis y valores

La propiedad **hyphens** en CSS controla la división de palabras al final de una línea cuando el texto se desborda.

Sintaxis:

```
etiqueta html {  
  hyphens: auto | manual | none;  
}
```

Valores:

- **auto**. El navegador decide dónde dividir las palabras para evitar el desbordamiento del texto. Utiliza las reglas de separación de sílabas del idioma predeterminado del documento o del sistema operativo.
- **manual**. Las palabras no se dividirán automáticamente. Se debe utilizar el carácter de guion (-) para indicar dónde se pueden dividir las palabras manualmente.
- **none**. Desactiva la división automática y manual de palabras. El texto puede desbordarse si no cabe en el contenedor.

Texto sin hyphens: Este es un texto largo que puede desbordar su contenedor si no se maneja correctamente. Cuando las palabras no se separan adecuadamente al final de la línea, la legibilidad del texto se ve afectada y el diseño general puede parecer desordenado.

Texto con hyphens: Este es un texto largo que puede desbordar su contenedor si no se maneja correctamente. Cuando las palabras no se separan adecuadamente al final de la línea, la legibilidad del texto se ve afectada y el diseño general puede parecer desordenado.



Ejemplo:

Por ejemplo, para permitir la división automática de palabras, podemos usar:

```
.hyphenation {  
  hyphens: auto; }
```

Esto permitirá que el navegador decida dónde dividir las palabras para evitar el desbordamiento del texto.

9.2 Definir el idioma para una correcta separación de sílabas

Además de la sintaxis y los valores de la propiedad `hyphens`, es importante **definir el idioma del contenido** para que la división de palabras funcione correctamente. Esto se logra utilizando la propiedad `lang` en HTML para especificar el idioma del texto y proporcionar al navegador la información necesaria para aplicar las reglas de separación de sílabas adecuadas.

Por ejemplo, si el texto está en español, debes incluir la siguiente declaración en el elemento `<html>`:

```
<html lang="es">
```

Esto indica que el contenido del documento está en español, lo que permite al navegador utilizar las reglas de separación de sílabas específicas del idioma español al dividir las palabras al final de una línea.

9.3 Fragmentos de texto en otro idioma al predefinido

Es importante destacar que si tenemos fragmentos de código o **texto en otro idioma dentro de tu documento HTML**, podemos **especificar el idioma correspondiente utilizando la etiqueta `lang` en esos elementos específicos**. Por ejemplo, si tenemos una cita o un fragmento de texto en inglés dentro de un documento en español, podemos agregar `lang="en"` al elemento que contiene ese texto en inglés.

```
<p lang="en">This is an example sentence in English within a  
paragraph.</p>
```

De esta manera, el navegador puede aplicar las reglas de separación de palabras adecuadas para ese idioma específico, asegurando una división de palabras correcta y coherente en todo el contenido del sitio web, independientemente del idioma en que esté escrito.

9.4 Ejemplos de uso

9.4.1 Usando hyphens en cards

El siguiente código crea una card con una imagen, un título (**h3**) y un párrafo (**p**). Ambos elementos de texto (**h3** y **p**) tienen la propiedad **hyphens** establecida en **auto**, lo que permite que el navegador divida automáticamente las palabras al final de una línea si es necesario para evitar el desbordamiento del contenedor. Esto mejora la legibilidad del texto, especialmente en dispositivos con pantallas más pequeñas donde el espacio es limitado.

```
HTML
1 * <div class="card">
2   
5   <h3>Lorem ipsum dolor sit amet,
6     consectetur adipiscing elit.</h3>
7   <p>Ut enim ad minim veniam, quis
8     nostrud exercitation ullamco laboris nisi
9     ut aliquip ex ea commodo consequat. Duis
10    aute irure dolor in reprehenderit in
11    voluptate velit esse cillum dolore eu
12    fugiat nulla pariatur. </p>
13 </div>

CSS
1 * body {
2   font-family: Arial, sans-serif;
3   margin: 0;
4   padding: 0;
5   background-color: #f4f4f4;
6 }
7
8 * .card {
9   max-width: 300px;
10  background-color: #fff;
11  border-radius: 8px;
12  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
13  margin: 20px auto;
14  padding: 20px;
15 }
16
17 * .card img {
18   width: 100%;
19   border-radius: 8px;
20   margin-bottom: 10px;
21 }
22
23 * .card h3 {
24   font-size: 22px;
25   margin-bottom: 10px;
26   hyphens: auto;
27 }
28
29 * .card p {
30   font-size: 18px;
31   line-height: 1.5;
32   hyphens: auto;
33 }
```



9.4.2 Usando hyphens con line-clamp

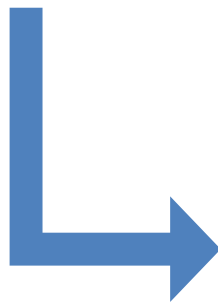
La combinación de las propiedades **hyphens** y **line-clamp** puede ser especialmente útil para controlar la legibilidad del texto en diseños responsivos. **Hyphens** se encarga de dividir las palabras al final de una línea, mientras que **line-clamp** **trunca el texto que excede un número específico de líneas** y muestra puntos suspensivos al final del texto truncado.

En el siguiente ejemplo mostramos un extracto de texto en una card con un máximo de tres líneas, y conseguimos que las palabras se dividan automáticamente al final de una línea si es necesario.



```
HTML
1 * <div class="card">
2   
5   <div class="texto">
6     <h3>Título de la card</h3>
7     <p>Este es un texto largo que puede
8       desbordar su contenedor si no se utiliza
9       hyphens. Cuando las palabras no se separan
10      adecuadamente al final de la línea, la
11      legibilidad del texto se ve afectada y el
12      diseño general puede parecer desordenado.
13    </p>
14  </div>
15 </div>

CSS
1 * .card {
2   border: 1px solid #ccc;
3   border-radius: 8px;
4   overflow: hidden;
5   width: 200px;
6 }
7
8 * .texto {
9   padding: 10px;
10 }
11
12 * .texto h3 {
13   margin-bottom: 10px;
14 }
15
16 * .texto p {
17   display: -webkit-box;
18   -webkit-line-clamp: 3; /* Mostrar solo 3 líneas de texto */
19   -webkit-box-orient: vertical;
20   overflow: hidden;
21   hyphens: auto; /* Dividir palabras al final de línea */
22 }
```





10. Display

La propiedad **display** permite especificar si un elemento se muestra o no y en caso de hacerlo, cómo se mostrará.

Todo elemento HTML tiene un valor de **display por defecto** que, dependiendo del elemento de qué se trate, este valor podrá ser **block** o **inline**.

- Los elementos de **bloque** son los que al renderizarse lo hacen en una **nueva línea** y utilizan todo el ancho disponible como, por ejemplo:
 - `<div>`
 - `<h1> .. <h6>`
 - `<p>`
 - `<form>`
 - `<header>`
 - `<section>`
- Los **elementos de línea** no comienzan una nueva línea y solo utilizan el espacio necesario cuando se crean como, por ejemplo:
 - `<span>`
 - `<a>`
 - `<img>`

Para **saltarse el display por defecto** solo hay que especificar en la propiedad **display** cómo queremos que se muestre el elemento en cuestión. Algunos de los valores que puede tomar dicha propiedad son:

- **inline**: por defecto para `<span>`. Se comporta como un solo bloque de línea. No permite modificar ancho y alto. **No se le puede aplicar las propiedades width, height ni margin**, ya que aunque tengamos mucho texto, se considera como una sola línea.
- **block**: permite modificar ancho y alto, pero añade salto de línea (líneas independientes). **Se aceptan las propiedades width, height y márgenes verticales**.

- **inline-block**: su comportamiento es una mezcla entre los dos anteriores, ya que los elementos se muestran en la **misma línea** (respetando el flujo y sin salto de línea) y se aceptan las propiedades **width, height y márgenes verticales**.

Es precisamente una de las características del **responsive design**, ya que en este caso lo que queremos es que los elementos se pongan unos al lado o debajo de los otros según el tamaño de la pantalla.

- **none**: permite **ocultar** un elemento y **no muestra el espacio reservado**. Este valor también se aplica al **responsive design**, ya que puede que nos interese ocultar algunos elementos si el tamaño de la pantalla es pequeño.

No confundir con **visibility:hidden**, el cual también oculta al elemento, pero sigue ocupando espacio en la página.

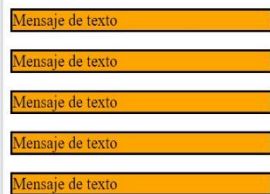


A continuación, se presentan algunos **ejemplos**:

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  border: 2px solid black;
  background: orange;
  display: block; }
</style>
</head>

<body>
<h2> Display: block </h2>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
</body>
</html>
```

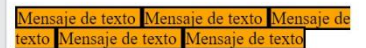
Display: block



```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  border: 2px solid black;
  background: orange;
  display: inline; }
</style>
</head>

<body>
<h2> Display: inline </h2>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
</body>
</html>
```

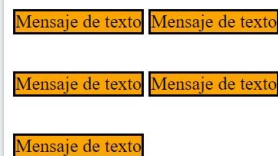
Display: inline



```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  border: 2px solid black;
  background: orange;
  display: inline-block; }
</style>
</head>

<body>
<h2> Display: inline-block </h2>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
<p> Mensaje de texto </p>
</body>
</html>
```

Display: inline-block





11. Posicionar o Position

La propiedad *position* de CSS establece en qué punto de la página comenzará a posicionarse o a mostrarse el elemento HTML que hayamos establecido en el código.

Es importante saber en qué punto se va a comenzar a “pintar” porque eso es lo que dirá al 100%, qué espacio ocupará cada elemento.

Existen **5 posibles** y diferentes **valores** que puede adoptar *position*:

- *static*
- *relative*
- *fixed*
- *absolute*
- *sticky*

Los elementos se posicionan entonces utilizando las **coordenadas** *top*, *bottom*, *left*, y *right*. Sin embargo, **estas coordenadas no funcionarán a menos que se establezca primero la propiedad *position***. También funcionan de manera diferente dependiendo del valor de la posición.

11.1 Static

Los elementos HTML se posicionan de **forma estática por defecto**. Es la posición natural de los elementos.

Los elementos posicionados estáticamente **no se ven afectados por las coordenadas *top*, *bottom*, *left* y *right***.

Un elemento con *position: static*; no se posiciona de ninguna manera especial; siempre **se posiciona según el flujo normal de la página**.

A continuación, se muestra un ejemplo.



```
<!DOCTYPE html>
<html>
<head>
<style>
.padre {
  width: 250px;
  height: 250px;
  margin: 25px;
  background: orange;
}

.hijo {
  width: 50px;
  height: 50px;
  background: red;
  position: static;
}

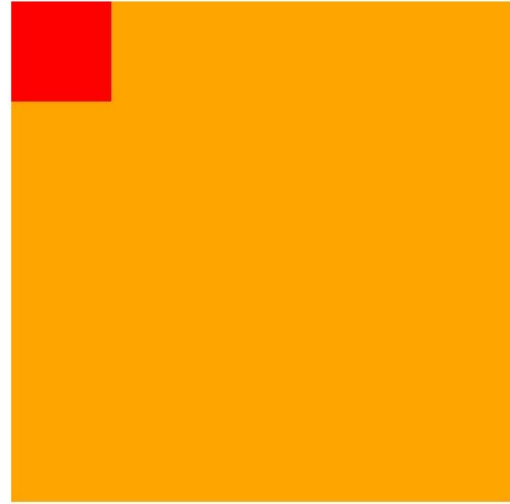
</style>
</head>
<body>

<h2>position: static;</h2>

<div class="padre">
  <div class="hijo"></div>
</div>

</body>
</html>
```

position: static;



11.2Relative

Un elemento con *position: relative;* se posiciona en relación a su posición normal.

Por tanto, podemos “relativizar” un elemento hijo con respecto a su padre, e indicarle en qué punto se tiene que “pintar” con respecto al origen. Esto se hace con las coordenadas *top*, *bottom*, *left* y *right*.

El resto del contenido no se ajustará para encajar en el hueco dejado por el elemento.

A continuación, se muestran algunos ejemplos.



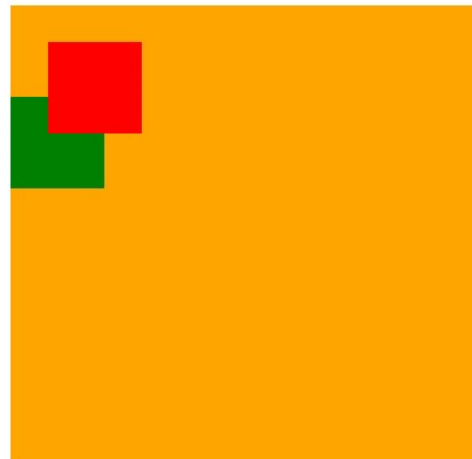
```
.padre {  
  width: 250px;  
  height: 250px;  
  margin: 25px;  
  background: orange;  
}  
  
.hijo {  
  width: 50px;  
  height: 50px;  
  background: red;  
  position: relative;  
  top: -30px;  
  left: 200px;  
}
```

position: relative;



```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
  .padre {  
    width: 250px;  
    height: 250px;  
    margin: 25px;  
    background: orange;  
  }  
  
  .hijo1 {  
    width: 50px;  
    height: 50px;  
    background: red;  
    position: relative;  
    top: 20px;  
    left: 20px;  
  }  
  
  .hijo2 {  
    width: 50px;  
    height: 50px;  
    background: green;  
  }  
</style>  
</head>  
<body>  
  
<h2>position: fixed;</h2>  
  
<div class="padre">  
  <div class="hijo1"></div>  
  <div class="hijo2"></div>  
</div>  
  
</body>  
</html>
```

position: relative;



11.3Fixed

Un elemento con *position: fixed;* se posiciona en relación con la ventana gráfica (o **body**), lo que significa que siempre se mantiene en el **mismo lugar** aunque se desplace la página. Las coordenadas *top*, *right*, *bottom* y *left* se utilizan para posicionar el elemento.

A continuación, se muestra un ejemplo:



```
<!DOCTYPE html>
<html>
<head>
<style>
.padre {
  width: 250px;
  height: 250px;
  margin: 25px;
  background: orange;
}

.hijo1 {
  width: 50px;
  height: 50px;
  background: red;
  position: fixed;
  top: 0px;
  left: 0px;
}

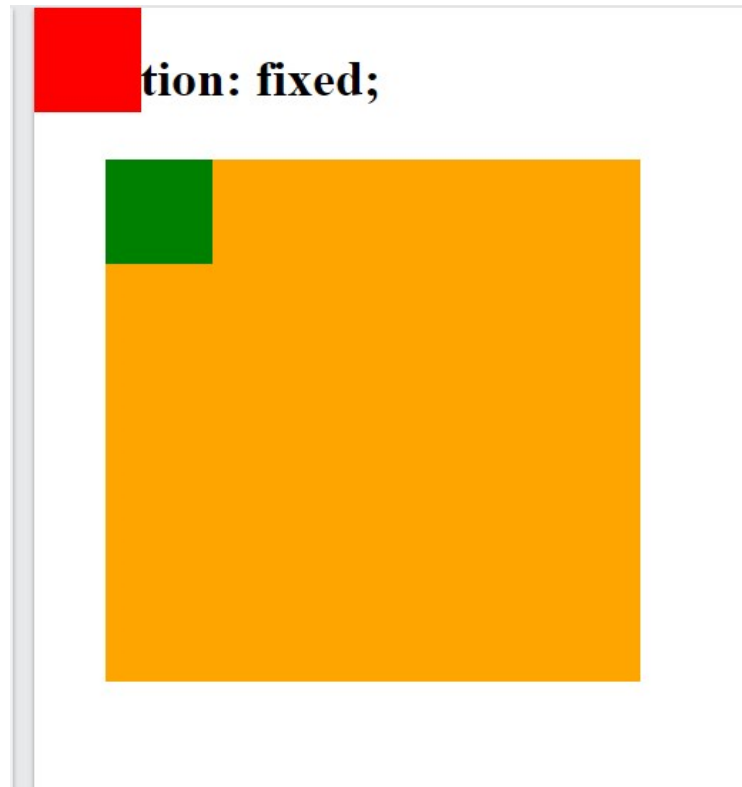
.hijo2 {
  width: 50px;
  height: 50px;
  background: green;
}

</style>
</head>
<body>

<h2>position: fixed;</h2>

<div class="padre">
  <div class="hijo1"></div>
  <div class="hijo2"></div>
</div>

</body>
</html>
```



Un elemento con **position: fixed;** se queda fijado y **no se mueve nunca**. Esto es muy útil, por **ejemplo**, en algunos menús en donde queremos que se queden en la parte superior mostrando siempre las opciones de dicho menú, o para anunciar algún producto, o para poner un formulario de suscripción, etc. Lo único que hay que tener en cuenta es que **no moleste al usuario**, ya que si es de un tamaño muy grande puede ocasionar un **efecto negativo** en la UX.

11.4 Absolute

Un elemento con **position: absolute;** se posiciona en relación con el ancestro posicionado más cercano que tenga una posición distinta a **static** (en lugar de posicionarse en relación con la ventana gráfica, como el fijo).

Los elementos posicionados absolutos son eliminados del flujo normal, y pueden superponerse a los elementos.



```
<!DOCTYPE html>
<html>
<head>
<style>
.padre {
  position: relative;
  width: 250px;
  height: 250px;
  margin: 25px;
  background: orange;
}

.hijo1 {
  width: 50px;
  height: 50px;
  background: red;
  position: absolute;
  top: 20px;
  left: 20px;
}

.hijo2 {
  width: 50px;
  height: 50px;
  background: green;
}

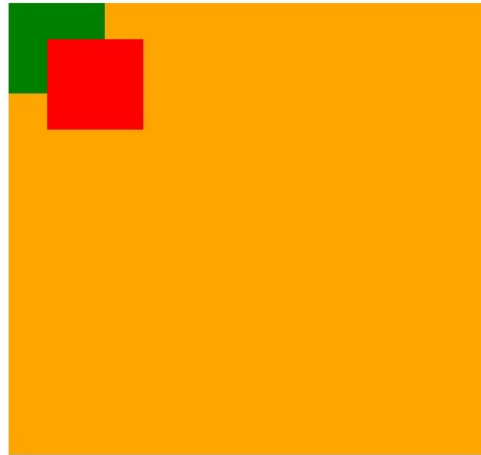
</style>
</head>
<body>

<h2>position: absolute;</h2>

<div class="padre">
  <div class="hijo1"></div>
  <div class="hijo2"></div>
</div>

</body>
</html>
```

position: absolute;



Sin embargo, **si un elemento con posición absoluta no tiene ancestros posicionados, utiliza el cuerpo del documento** y se mueve con el desplazamiento de la página.

```
<!DOCTYPE html>
<html>
<head>
<style>
div.padre {
  width: 400px;
  height: 200px;
  background: orange;
}

div.hijo {
  position: absolute;
  top: 50px;
  right: 0;
  width: 200px;
  height: 100px;
  background: LightCoral;
}

</style>
</head>
<body>

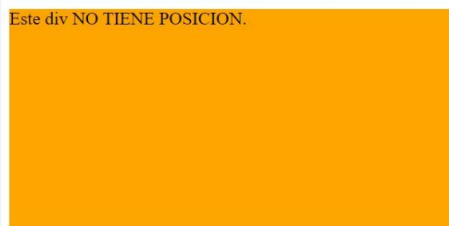
<h2>position: absolute; </h2>

<div class="padre">Este div NO
TIENE POSICION.
  <div class="hijo">Este div
TIENE POSICION position:
absolute;</div>
</div>

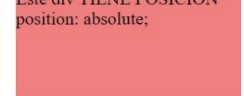
</body>
</html>
```

position: absolute;

Este div NO TIENE POSICION.



Este div TIENE POSICION
position: absolute;





11.5 Sticky

Un elemento con ***position: sticky;*** se posiciona en función de la posición del **scroll** del usuario. Por tanto, los elementos son posicionados **de forma relativa hasta que su bloque contenedor alcanza un límite** establecido.

Un elemento ***sticky*** **alterna entre relative y fixed**, dependiendo de la posición del scroll. Se posiciona de forma relativa hasta que se alcanza una determinada posición de desplazamiento en la ventana gráfica - entonces se "pega" en su lugar (como ***position: fixed;***).

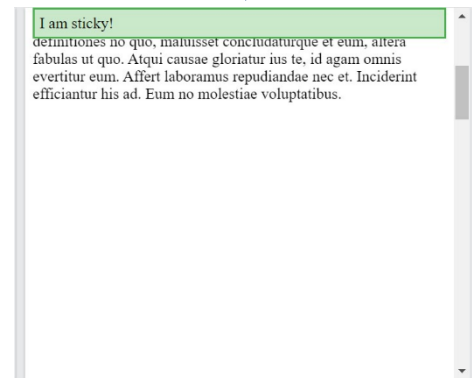
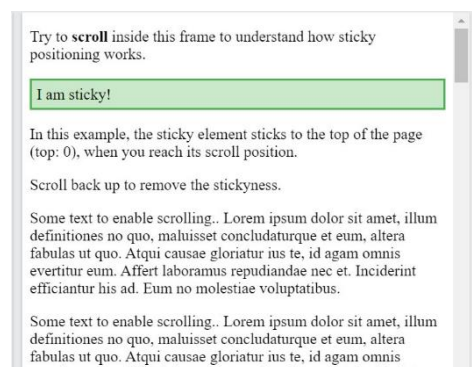
```
<!DOCTYPE html>
<html>
<head>
<style>
div.sticky {
  position: -webkit-sticky;
  position: sticky;
  top: 0;
  padding: 5px;
  background-color: #cae8ca;
  border: 2px solid #4CAF50;
}
</style>
</head>
<body>

<p>Try to <b>scroll</b> inside this frame to understand how sticky
positioning works.</p>

<div class="sticky">I am sticky!</div>

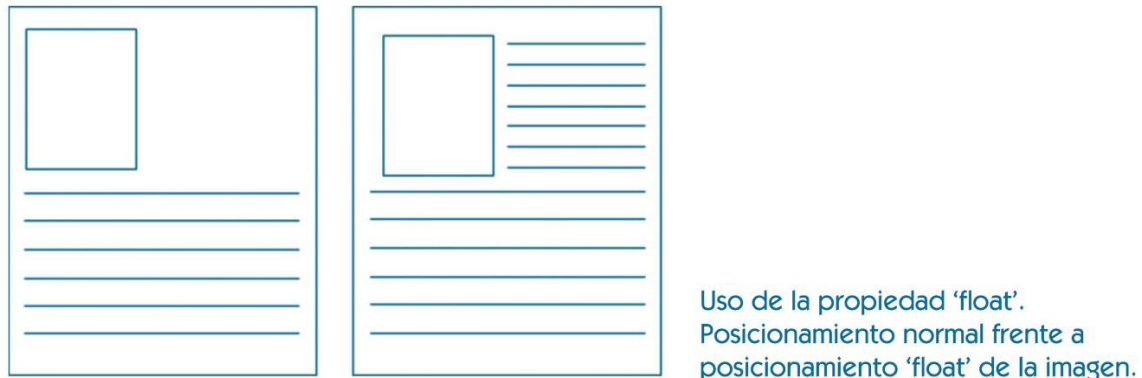
<div style="padding-bottom:2000px">
  <p>In this example, the sticky element sticks to the top of the page
(top: 0), when you reach its scroll position.</p>
  <p>Scroll back up to remove the stickiness.</p>
  <p>Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum
definitiones no quo, malisset concludaturque et eum, altera fabulas ut
quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert
laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no
molestiae voluptatibus.</p>
  <p>Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum
definitiones no quo, malisset concludaturque et eum, altera fabulas ut
quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert
laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no
molestiae voluptatibus.</p>
</div>

</body>
</html>
```



12. Float y Clear

Cuando se trabaja a través de cajas para el modelado del código de la página web, **debe tenerse en cuenta la posición** de cada una de ellas. Normalmente, estas se sitúan la una a continuación de la otra, bien en horizontal o en vertical, pero existen ocasiones en las que es deseable que diversas cajas contenedoras de información, ya sea texto, imagen o cualquier otro elemento, **se adapten entre sí**.



En la figura de la izquierda, se observa que en su parte izquierda aparece una fotografía y, a continuación, el texto asociado. Ahora bien, se puede observar cómo en la figura de la derecha, texto e imagen se funden ofreciendo una mejor estética a la web. En este segundo caso, se está utilizando la propiedad de flotación, es decir, **'float'**.

La existencia de cajas flotantes condiciona la situación del resto de cajas. Esto es debido a que aquellos elementos no flotantes, es decir, los que están en línea, adaptan su posición para evitar que se solapen las cajas en función a la colocación de las flotantes.

El **funcionamiento** concreto de esta propiedad se basa en el desplazamiento total hacia la derecha o hacia la izquierda de una caja contenedora con respecto de su posición inicial.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    float: left | right | none;  
}
```

Los tres posibles valores que puede tomar esta propiedad son hacia la **izquierda** (**left**), hacia la **derecha** (**right**) o mantenerse en la **posición original** (**none**). Este último es el valor por defecto y es usado en aquellos casos en los que se desea eliminar la propiedad

‘float’ definida para una caja. En los casos **left** y **right**, el resto de los elementos se adaptan para mostrarse respectivamente a la derecha o a la izquierda de la caja flotante; si no se desea que el resto de los elementos tomen este papel, se utilizará la propiedad ‘**clear**’.

La propiedad ‘**clear**’ establece si un elemento debe estar al lado de los elementos flotantes que lo preceden o si debe situarse bajo ellos. Se suele utilizar para **restaurar el flujo normal del documento** y así los elementos dejan de flotar hacia la izquierda, la derecha o ambos lados.

Ejemplos:

```
<!DOCTYPE html>
<html>
<head>
<style>
img {
  float: left;
  margin-right: 2em;
  margin-bottom: .3em;
}
</style>
</head>
<body>

<h2>Float Left</h2>

<p>
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus
imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae
scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices
nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut
aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio.
Praesent convallis urna a lacus interdum ut hendrerit risus congue.
Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at
libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui
eget tellus gravida venenatis. Integer fringilla congue eros non
fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus.
Mauris quis diam velit.</p>

</body>
</html>
```

Float Left



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula,

facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus. Mauris quis diam velit.

```
<!DOCTYPE html>
<html>
<head>
<style>
img {
  float: none;
}
</style>
</head>
<body>

<h2>Float none</h2>

<p>
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus
imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae
scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices
nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut
aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio.
Praesent convallis urna a lacus interdum ut hendrerit risus congue.
Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at
libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui
eget tellus gravida venenatis. Integer fringilla congue eros non
fermentum. Sed dapibus pulvinar nibh tempor porta. Cras ac leo purus.
Mauris quis diam velit.</p>

</body>
</html>
```

Float none



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac. In at libero sed nunc venenatis imperdiet sed ornare turpis. Donec vitae dui eget tellus gravida venenatis. Integer fringilla congue eros non



```
<!DOCTYPE html>
<html>
<head>
<style>
.div1 {
  float: left;
  padding: 10px;
  border: 3px solid #73AD21;
}

.div2 {
  padding: 10px;
  border: 3px solid red;
}

.div3 {
  float: left;
  padding: 10px;
  border: 3px solid #73AD21;
}

.div4 {
  padding: 10px;
  border: 3px solid red;
  clear: left;
}
</style>
</head>
<body>

<h2>Without clear</h2>
<div class="div1">div1</div>
<div class="div2">div2 - Notice that div2 is after div1 in the HTML
code. However, since div1 floats to the left, the text in div2
flows around div1.</div>
<br><br>

<h2>With clear</h2>
<div class="div3">div3</div>
<div class="div4">div4 - Here, clear: left; moves div4 down below the
floating div3. The value "left" clears elements floated to the left.
You can also clear "right" and "both".</div>

</body>
</html>
```

Without clear

div1 div2 - Notice that div2 is after div1 in the HTML code. However, since div1 floats to the left, the text in div2 flows around div1.

With clear

div3 div4 - Here, clear: left; moves div4 down below the floating div3. The value "left" clears elements floated to the left. You can also clear "right" and "both".

12.1 Problemas con flotantes en CSS y sus soluciones

El uso de **flotantes** (float) en CSS ha sido una técnica común para el diseño de layout antes de la introducción de **Flexbox** y **CSS Grid**. Sin embargo, trabajar con flotantes puede llevar a varios errores o problemas comunes, especialmente en lo que respecta al **flujo del documento y la contención de elementos**. Veamos algunos ejemplos de estos **errores y cómo solucionarlos**.

1) Problema del contenedor que no contiene sus flotantes

- **Problema:** el contenedor `.container` no se expande para incluir los elementos flotantes, por lo que el fondo no se muestra cómo se espera.
- **Solución:** utilizar la técnica del `"clearfix"` o `overflow: hidden`.

La técnica del `"clearfix"` utiliza un pseudo-elemento (`::after`) para insertar contenido vacío después del último elemento flotante dentro del contenedor. La propiedad `display: block`; asegura que este pseudo-elemento se comporte como un elemento de bloque, y `clear: both`; fuerza al pseudo-elemento a moverse debajo

de ambos flotantes (tanto izquierdo como derecho). Esto efectivamente “limpia” los flotantes y hace que el contenedor padre reconozca la altura total de sus elementos hijos flotantes.

```
HTML
1 <h1>Problema del contenedor que no contiene sus
  flotantes</h1>
2 <div class="container">
3   <div class="floating">Elemento Flotante 1</div>
4   <div class="floating">Elemento Flotante 2</div>
5 </div>

CSS
1 /* Descripción del problema: El contenedor no se expande para contener
  los flotantes por eso no se ve el .container de color lightblue*/
2 .container {
3   background-color: lightblue;
4   /* Solución 1: overflow:hidden;*/
5   /*overflow:hidden;*/
6 }
7 .floating {
8   float: left;
9   width: 50%;
10 }
11 /* Solución 2: limpiamos los flotantes con clear:both; */
12 /*.container::after {
13   content: "";
14   display: block;
15   clear: both;
16 }*/
```

- Si NO aplicamos ninguna de las soluciones, el resultado será el siguiente:

Problema del contenedor que no contiene sus flotantes

Elemento Flotante 1

Elemento Flotante 2

- Si aplicamos alguna de las dos soluciones, el resultado será el siguiente:

Problema del contenedor que no contiene sus flotantes

Elemento Flotante 1

Elemento Flotante 2

2) Problema de colapsamiento de contenedores

- **Problema:** al definir de color lightblue el párrafo (<p>) que hay a continuación de los flotantes, se ponen de color azul también los flotantes.
- **Solución:** asegurarse de limpiar los flotantes.

```
HTML
1 <h1>Problema de colapsamiento de contenedores</h1>
2 <div class="container">
3   <div class="floating">Elemento Flotante 1</div>
4   <div class="floating">Elemento Flotante 2</div>
5 </div>
6 <p>Un párrafo con color de fondo lightblue a
  continuación de flotantes.</p>
7

CSS
1 /* Descripción del problema: Al definir de color lightblue el párrafo
  que hay a continuación de los flotantes, se ponen de color azul
  también los flotantes*/
2 .floating {
3   float: left;
4   width: 50%;
5 }
6 .container {
7   overflow: hidden;
8 }
9 <p> {
10 /*Solución: limpiamos los flotantes para que el párrafo se vea de
  color azul con clear:both; y los flotantes no tengan ese color*/
11 /*clear:both;*/
12 background-color: lightblue;
13 }
```



- Resultado sin la aplicación de ninguna solución:

Problema de colapsamiento de contenedores

Elemento Flotante 1	Elemento Flotante 2
Un párrafo con color de fondo lightblue a continuación de flotantes.	

- Si aplicamos la solución propuesta, el resultado será el siguiente:

Problema de colapsamiento de contenedores

Elemento Flotante 1	Elemento Flotante 2
Un párrafo con color de fondo lightblue a continuación de flotantes.	

3) Problema de alineamiento de flotantes

- **Problema:** flotantes de diferentes alturas pueden causar un alineamiento irregular.
- **Solución:** asegurar que todos los flotantes tengan la misma altura (sería definiendo un alto fijo) o considerar usar Flexbox/CSS Grid para un diseño más uniforme.

```
HTML
1 <div class="container">
2   <div class="floating tall">Elemento Flotante con mucho
   texto: Lorem ipsum dolor sit amet consectetur adipiscing elit
   taciti malesuada odio rhoncus, metus lacus volutpat pulvinar
   interdum montes sed potenti integer sociis massa, turpis quis
   cras vivamus mi blandit pretium iaculis aptent velit. Arcu
   semper malesuada euismod scelerisque netus faucibus fringilla,
   montes viverra accumsan dis eget integer dictumst, libero eros
   ante commodo duis quam. Nisl cras nec laoreet pulvinar ut
   aliquet vehicula congue, malesuada felis erat risus velit
   viverra luctus sollicitudin, sagittis ridiculus eleifend lectus
   euismod vulputate a.</div>
3   <div class="floating short">Elemento Flotante Bajo</div>
4 </div>
5

CSS
1 .floating {
2   box-sizing: border-box;
3   float: left;
4   width: 50%;
5   background-color: lightblue;
6   padding: 10px;
7   border: 3px solid lightgrey;
8   overflow: hidden;
9 }
10
11 /*Solución: asignar la misma altura a los
   flotantes */ /*
12 .tall {
13   height: 150px;
14 }
15
16 .short {
17   height: 150px;
18 } */
```

- Resultado sin la aplicación de ninguna solución:

Elemento Flotante con mucho texto: Lorem ipsum dolor sit amet consectetur adipiscing elit taciti malesuada odio rhoncus, metus lacus volutpat pulvinar interdum montes sed potenti integer sociis massa, turpis quis cras vivamus mi blandit pretium iaculis aptent velit. Arcu semper malesuada euismod scelerisque netus faucibus fringilla, montes viverra accumsan dis eget integer dictumst, libero eros ante commodo duis quam. Nisl cras nec laoreet pulvinar ut aliquet vehicula congue, malesuada felis erat risus velit viverra luctus sollicitudin, sagittis ridiculus eleifend lectus euismod vulputate a.	Elemento Flotante Bajo
--	------------------------

- Si aplicamos la solución propuesta, el resultado será el siguiente:

Elemento Flotante con mucho texto: Lorem ipsum dolor sit amet consectetur adipiscing elit taciti malesuada odio rhoncus, metus lacus volutpat pulvinar interdum montes sed potenti integer sociis massa, turpis quis cras vivamus mi blandit pretium iaculis aptent velit. Arcu semper malesuada euismod scelerisque netus faucibus fringilla, montes viverra accumsan dis eget integer dictumst, libero eros ante commodo duis quam. Nisl cras nec laoreet pulvinar ut aliquet vehicula congue, malesuada felis erat risus velit viverra	Elemento Flotante Bajo
---	------------------------



13. Propiedades de visibilidad

En este apartado veremos algunas propiedades CSS relacionadas con la visibilidad de los elementos HTML en nuestra interfaz web. Estos son: *z-index*, *overflow*, y *visibility*.

13.1 'z-index'

La propiedad *z-index* indica la posición de un elemento en el eje z, es decir, la **profundidad** que tiene un elemento respecto a otro de cara al usuario. Vendría a ser el equivalente a las capas de Photoshop.

z-index sólo funciona en **elementos posicionados** (*position: absolute*, *position: relative*, *position: fixed* o *position: sticky*) y **elementos flex** (elementos que son hijos directos de elementos *display: flex*).

Si dos elementos posicionados se **superponen** sin un *z-index* especificado, el elemento posicionado en último lugar en el código HTML es el que se mostrará en la parte superior.

Su **sintaxis** es la siguiente:

```
etiqueta html {  
    z-index: auto | number;  
}
```

- *auto*: establece el z-index igual al de sus padres. Es el valor por defecto.
- *number*: establece la posición del elemento en el eje z en número. Se permiten números negativos. Si no se especifica el valor, tendrá un valor de 0. **Cuanto mayor es su valor, más al frente tendremos al elemento.**

Recomendación: enumerar las capas de z-index de 10 en 10, ya que así evitamos modificar los valores si introducimos una capa nueva en medio.

Ejemplo:

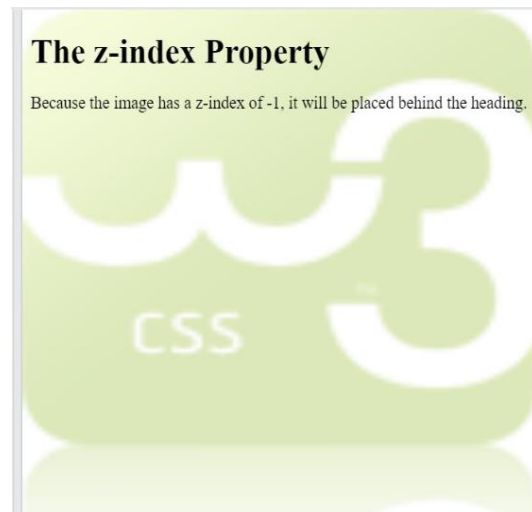
```
<!DOCTYPE html>
<html>
<head>
<style>
img {
  position: absolute;
  left: 0px;
  top: 0px;
  z-index: -1;
  opacity: .3
}
</style>
</head>
<body>

<h1>The z-index Property</h1>



<p>Because the image has a z-index of -1, it will be placed behind the heading.</p>

</body>
</html>
```



13.2 'overflow'

La propiedad CSS *overflow* controla lo que ocurre con el contenido que es demasiado grande para caber en un área. Por tanto, **especifica si se recorta el contenido o se añaden barras de desplazamiento cuando el contenido de un elemento es demasiado grande para caber en el área especificada.**

overflow sólo funciona **para elementos de bloque con una altura especificada.**

La propiedad *overflow* tiene los siguientes valores:

- *visible* - Por defecto. El desbordamiento no se recorta. El contenido se muestra fuera de la caja del elemento.
- *hidden* - El desbordamiento se recorta, y el resto del contenido será invisible. Se utiliza mucho en animaciones.
- *scroll* - El desbordamiento se recorta y se añade una barra de desplazamiento para ver el resto del contenido.
- *auto* - Similar a scroll, pero añade barras de desplazamiento sólo cuando es necesario.

A continuación, se presentan algunos **ejemplos**:



```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  background-color: orange;
  width: 200px;
  height: 65px;
  border: 1px solid;
  overflow: visible;
}
</style>
</head>
<body>
```

<h2>Overflow: visible</h2>

<div>You can use the overflow property when you want to have better control of the layout. The overflow property specifies what happens if content overflows an element's box.</div>

```
</body>
</html>
```

Overflow: visible

You can use the overflow property when you want to have better control of the layout. The overflow property specifies what happens if content overflows an element's box.

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  background-color: orange;
  width: 200px;
  height: 65px;
  border: 1px solid;
  overflow: hidden;
}
</style>
</head>
<body>
```

<h2>Overflow: hidden</h2>

<div>You can use the overflow property when you want to have better control of the layout. The overflow property specifies what happens if content overflows an element's box.</div>

```
</body>
</html>
```

Overflow: hidden

You can use the overflow property when you want to have better control of the layout. The overflow property

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  background-color: orange;
  width: 200px;
  height: 80px;
  border: 1px solid;
  overflow: scroll;
}
</style>
</head>
<body>
```

<h2>Overflow: scroll</h2>

<div>You can use the overflow property when you want to have better control of the layout. The overflow property specifies what happens if content overflows an element's box.</div>

```
</body>
</html>
```

Overflow: scroll

You can use the overflow property when you want to have better control of the layout. The overflow

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  background-color: orange;
  width: 200px;
  height: 80px;
  border: 1px solid;
  overflow: auto;
}
</style>
</head>
<body>
```

<h2>Overflow: auto</h2>

<div>You can use the overflow property when you want to have better control of the layout. The overflow property specifies what happens if content overflows an element's box.</div>

```
</body>
</html>
```

Overflow: auto

You can use the overflow property when you want to have better control of the layout. The overflow

13.3 'visibility'

La propiedad **visibility** especifica si un elemento es **visible** o **no**. Si ocultamos elementos con esta propiedad, tenemos que tener en cuenta que **el flujo de posicionamiento de elementos no cambia**, simplemente no se muestra el elemento. Por tanto, **los elementos ocultos ocupan espacio** en la página.

Si queremos ocultar y eliminar un elemento en el diseño del documento, debemos utilizar la propiedad **display**.



Su **sintaxis** es la siguiente:

```
etiqueta html {  
    visibility: visible | hidden | collapse;  
}
```

- *visible*. Es el valor por defecto. El elemento es visible.
- *hidden*. El elemento está oculto (pero sigue ocupando espacio).
- *collapse*. Sólo para filas de tabla (`<tr>`), grupos de filas (`<tbody>`), columnas (`<col>`), grupos de columnas (`<colgroup>`). Este valor elimina una fila o una columna, pero no afecta al diseño de la tabla. El espacio ocupado por la fila o columna estará disponible para otros contenidos.
- Si se utiliza el *collapse* en otros elementos, se representa como "oculto".

Ejemplo:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
h2.a {  
    visibility: visible;  
}  
  
h2.b {  
    visibility: hidden;  
    background: yellow;  
}  
</style>  
</head>  
<body>  
  
<h1>La propiedad visibility</h1>  
  
<h2 class="a">Este heading es visible</h2>  
  
<h2 class="b">Este heading esta oculto</h2>  
  
<p>Fijate que el heading que esta oculto  
sigue ocupando espacio en la pagina.</p>  
  
</body>  
</html>
```

La propiedad visibility

Este heading es visible

Fijate que el heading que esta oculto sigue ocupando espacio en la pagina.